

Informe Técnico Final - DFS-Bloques

Sistema de archivos distribuidos por bloques - DFS-Bloques

Santiago Restrepo Salazar y Josue Pino Pino

Mayo de 2026

Universidad Pontificia Bolivariana

Facultad de Ingeniería de Sistemas e Informática

Arquitecturas de Nube y Sistemas Distribuidos

Profesor: Álvaro Ospina Sanjuan

Proyecto: Diseño e implementación de un sistema de archivos distribuido por bloques

Estudiante: Santiago Restrepo, Josue Pino

ID: 000435253 - 000208754

Ciudad: Medellín, Colombia

Tabla de contenido

1. Introducción
2. Objetivo general
3. Objetivos específicos
4. Marco teórico
5. Descripción del servicio y problema abordado
6. Alcance del proyecto
7. Requerimientos funcionales y no funcionales
8. Arquitectura del sistema
9. Especificación de protocolos y APIs
10. Algoritmos implementados
11. Implementación desarrollada y estructura del repositorio
12. Ambiente técnico y configuración
13. Entorno de ejecución final con Docker Compose y alternativa AWS Academy

14. Gestión del proyecto con Trello
15. Plan de pruebas
16. Resultados y análisis de pruebas
17. Problemas encontrados y ajustes realizados
18. Conclusiones
19. Referencias
20. Anexos y lista de evidencias pendientes

1. Introducción

El presente informe técnico final documenta el diseño, la implementación desarrollada y el plan de validación del proyecto DFS-Bloques, un sistema de archivos distribuido por bloques desarrollado en el contexto del curso Arquitecturas de Nube y Sistemas Distribuidos. El proyecto responde al objetivo de construir un DFS minimalista capaz de almacenar y recuperar archivos grandes en múltiples nodos, aplicando particionamiento en bloques, distribución de datos y replicación.

El sistema se plantea como una solución académica orientada a evidenciar conceptos centrales de los sistemas distribuidos: separación entre plano de control y plano de datos, comunicación entre procesos mediante red, tolerancia parcial a fallos, replicación, metadatos distribuidos lógicamente, exposición de APIs, ejecución en contenedores y portabilidad hacia entornos de nube. No se pretende construir un sistema de archivos integrado al kernel del sistema operativo ni una solución productiva equivalente a HDFS, GFS, NFS o un servicio de almacenamiento en la nube. El alcance se limita a una versión funcional y verificable que permita demostrar los mecanismos fundamentales de un DFS basado en bloques.

La arquitectura propuesta utiliza un modelo **Cliente/Servidor** combinado con un patrón **Maestro-Trabajador**. El componente maestro, denominado **NameNode**, administra los metadatos del sistema: usuarios, directorios, archivos, bloques, ubicación de réplicas y estado de los nodos de almacenamiento. Los componentes trabajadores, denominados **DataNodes**, almacenan físicamente los bloques como archivos binarios en carpetas locales o volúmenes de contenedor. El cliente interactúa con el sistema mediante una CLI o API mínima que permite ejecutar operaciones como put, get, ls, rm, mkdir y rmdir.

El enfoque técnico prioriza la claridad arquitectónica, la facilidad de prueba y la trazabilidad. Por esta razón, la comunicación se define mediante REST sobre HTTP, los mensajes de control se serializan en JSON y la transferencia de bloques se realiza mediante HTTP con contenido binario. Para facilitar el despliegue, el sistema se ejecuta en Docker Compose local como entorno principal de demostración, mientras que AWS Academy queda documentado como alternativa de despliegue mediante variables de configuración como URLs, puertos y rutas.

2. Objetivo general

Diseñar e implementar un sistema de archivos distribuido minimalista por bloques que permita cargar, distribuir, replicar, consultar, recuperar y eliminar archivos mediante un cliente CLI/API, un NameNode central para la gestión de metadatos y múltiples DataNodes encargados del almacenamiento físico de bloques.

3. Objetivos específicos

1. Definir una arquitectura distribuida basada en Cliente, NameNode y DataNodes.
2. Implementar un NameNode encargado de administrar usuarios, directorios, archivos, bloques, ubicaciones de réplicas y estado de nodos.
1. Implementar múltiples DataNodes capaces de recibir, almacenar, listar, entregar y eliminar bloques de archivos.
2. Implementar un cliente CLI/API que permita ejecutar operaciones básicas del DFS: put, get, ls, rm, mkdir y rmdir.
3. Dividir archivos en bloques de tamaño configurable antes de almacenarlos.
4. Replicar cada bloque en al menos dos DataNodes distintos para brindar tolerancia mínima a fallos.
5. Reconstruir archivos descargando sus bloques desde DataNodes disponibles y uniéndolos en el orden correcto.
6. Manejar errores básicos por credenciales inválidas, archivos inexistentes, DataNodes no disponibles, bloques faltantes o réplicas insuficientes.
7. Ejecutar el sistema en contenedores Docker para simular un clúster distribuido local.
8. Documentar una alternativa de despliegue en AWS Academy, manteniendo como entorno final de demostración el clúster local con Docker Compose validado con el profesor.
9. Documentar el proceso de diseño, implementación, pruebas, gestión de tareas y evidencias.

4. Marco teórico

4.1 Sistemas distribuidos

Un sistema distribuido está compuesto por múltiples procesos o nodos que cooperan a través de una red para ofrecer una funcionalidad común. Aunque internamente existen varios componentes independientes, el usuario espera interactuar con el sistema como si se tratara de una unidad coherente. Este tipo de sistemas debe considerar problemas que no aparecen de la misma forma en aplicaciones monolíticas locales: latencia, fallos parciales, pérdida de mensajes, heterogeneidad, sincronización, consistencia, seguridad, tolerancia a fallos y observabilidad.

En un sistema distribuido, cada proceso conserva su propio espacio de memoria. Por tanto, los componentes no comparten variables ni estructuras internas directamente. La cooperación se logra mediante mensajes, protocolos, llamadas remotas, sockets, HTTP, RPC, colas de mensajes u otros mecanismos de middleware. Esta característica obliga a definir contratos de comunicación claros, formatos de intercambio, códigos de error y estrategias de recuperación.

DFS-Bloques adopta esta visión: el cliente, el NameNode y los DataNodes son procesos separados. La comunicación entre ellos ocurre mediante red, y cada componente tiene responsabilidades específicas. El NameNode no accede directamente al almacenamiento interno del cliente y el cliente no modifica los

metadatos sin pasar por la API del NameNode. Esta separación permite demostrar la coordinación distribuida entre servicios independientes.

4.2 Sistemas de archivos distribuidos

Un sistema de archivos distribuido, o DFS, permite almacenar, acceder y administrar archivos cuyo contenido se encuentra físicamente repartido en diferentes nodos. En sistemas maduros, el usuario puede percibir una vista lógica de archivos y directorios sin conocer la ubicación real de los datos. La finalidad es aumentar disponibilidad, capacidad de almacenamiento, tolerancia a fallos y, en algunos escenarios, rendimiento agregado.

El problema central de un DFS consiste en responder varias preguntas:

- ¿Dónde se almacena cada parte de un archivo?
- ¿Cómo se localizan esas partes cuando un cliente quiere leer el archivo?
- ¿Cómo se mantiene la relación entre archivo lógico y datos físicos?
- ¿Cómo se recupera un archivo si un nodo de almacenamiento falla?
- ¿Cómo se evita que los metadatos indiquen que un archivo existe cuando sus bloques no fueron guardados correctamente?
- ¿Cómo se evidencia que los datos están distribuidos y replicados?

Para responder estas preguntas, DFS-Bloques separa el sistema en dos planos. El **plano de control** se ubica principalmente en el NameNode y gestiona metadatos, decisiones de asignación y estado de nodos. El **plano de datos** se ubica en los DataNodes y en el cliente, porque la transferencia de bloques se realiza directamente entre el cliente y los nodos de almacenamiento.

4.3 DFS basado en bloques

En un DFS basado en bloques, la unidad de distribución no es necesariamente el archivo completo, sino fragmentos de tamaño fijo o configurable llamados bloques. Un archivo grande puede dividirse en varios bloques, y cada bloque puede almacenarse en uno o más nodos. Esta estrategia permite distribuir archivos grandes entre varias máquinas, replicar porciones específicas, reconstruir archivos en orden y tolerar la caída de un nodo cuando existen réplicas disponibles.

El tamaño de bloque es una decisión de diseño relevante. Bloques muy pequeños aumentan la cantidad de metadatos y peticiones, mientras que bloques muy grandes reducen la cantidad de metadatos pero pueden afectar la distribución de archivos pequeños. En este proyecto se mantiene como valor de diseño un tamaño por defecto de **64 MB**, coherente con sistemas como GFS y con el enunciado del proyecto. Sin embargo, para las pruebas locales se propone usar tamaños menores, como 1 MB, con el fin de evidenciar visualmente la partición de archivos en múltiples bloques.

Cada bloque debe tener por lo menos:

- Un identificador único.
- Un número de orden dentro del archivo.
- Un tamaño real.
- Una o varias ubicaciones físicas.
- Opcionalmente, un checksum para validar integridad.

El orden de los bloques es crítico. Si el archivo se reconstruye con bloques desordenados, el archivo resultante será inválido. Por eso, el NameNode debe conservar el metadato order y el cliente debe respetarlo durante get.

4.4 Sistemas orientados a escritura una vez y lectura múltiple

El proyecto se acerca a un modelo **WORM**: escribir una vez y leer muchas veces. En este modelo, el sistema no se optimiza para modificar fragmentos arbitrarios de un archivo ya almacenado. En lugar de actualizaciones parciales, la operación principal es subir un archivo completo mediante put, almacenarlo como conjunto de bloques y recuperarlo completo mediante get.

Este enfoque reduce la complejidad de consistencia. Al no permitir modificaciones parciales concurrentes sobre bloques ya existentes, se evita el problema de coordinar múltiples escritores sobre el mismo archivo. Para un proyecto académico, esta restricción es adecuada porque permite concentrar el esfuerzo en particionamiento, distribución, replicación, metadatos y recuperación.

4.5 GFS como antecedente conceptual

Google File System (GFS) es un antecedente importante porque propone una arquitectura con un **master** central y múltiples servidores de chunks. Los archivos se dividen en chunks de tamaño grande, el master mantiene metadatos y los clientes interactúan con el master para localizar datos, pero transfieren la información directamente con los chunkservers. Esta separación entre control y datos es uno de los principios adoptados en DFS-Bloques.

GFS fue diseñado para archivos grandes, fallos frecuentes de componentes y cargas de trabajo intensivas en datos. Una idea clave es que las fallas de componentes deben asumirse como normales y no como excepcionales. Por ello, el sistema debe monitorear nodos, detectar errores, replicar datos y recuperarse de forma controlada. DFS-Bloques adopta una versión simplificada de esta idea: cada bloque se replica en dos DataNodes, y el cliente intenta descargar desde otra réplica si un DataNode no responde.

Otra idea relevante de GFS es minimizar la participación del master en las transferencias de datos. Si todos los bytes pasaran por el master, este podría convertirse en cuello de botella. Por eso, en DFS-Bloques, el NameNode entrega ubicaciones y asignaciones, pero los bloques viajan directamente entre cliente y DataNodes.

4.6 HDFS como antecedente conceptual

Hadoop Distributed File System (HDFS) también es un referente directo para DFS-Bloques. HDFS organiza el sistema mediante un NameNode que administra metadatos y DataNodes que almacenan bloques. El cliente consulta al NameNode para conocer la ubicación de los bloques y luego accede a los DataNodes para leer o escribir datos.

DFS-Bloques no implementa todas las capacidades de HDFS. No incluye alta disponibilidad del NameNode, re-replicación completa en segundo plano, balanceo sofisticado, permisos avanzados, integración con Hadoop ni optimizaciones de lectura distribuida. Sin embargo, adopta la estructura conceptual mínima: NameNode, DataNodes, archivos divididos en bloques, replicación y recuperación por ubicaciones.

4.7 NameNode y metadatos

El NameNode es el componente maestro del sistema. Su responsabilidad es mantener la información lógica necesaria para operar el DFS. No almacena los archivos completos ni los bloques como datos principales. En cambio, conserva la relación entre usuarios, directorios, archivos, bloques y ubicaciones.

Los metadatos principales son:

- Usuarios registrados.
- Directorios lógicos por usuario.
- Archivos por usuario y ruta.
- Tamaño del archivo.
- Tamaño de bloque usado.
- Cantidad total de bloques.
- Lista de bloques con su orden.
- Ubicaciones de cada bloque.
- Estado de DataNodes.
- Estado de disponibilidad del archivo.

El NameNode permite que el cliente no tenga que conocer la distribución interna de los datos. Para put, indica dónde guardar cada bloque. Para get, informa dónde recuperar cada bloque. Para ls, presenta la vista lógica de archivos del usuario. Para rm, permite eliminar metadatos y bloques asociados.

4.8 DataNodes y almacenamiento físico

Los DataNodes son los nodos trabajadores del sistema. Su función es almacenar bloques como archivos binarios en una ruta local. Cada DataNode expone una API REST con operaciones para verificar estado, guardar bloques, listar bloques, entregar bloques y eliminarlos.

En la demostración local, cada DataNode se ejecuta como un contenedor independiente y utiliza un volumen separado. Esto permite evidenciar físicamente que los bloques no se almacenan todos en una misma ubicación. Por ejemplo:

```
storage/datanode1/  
storage/datanode2/  
storage/datanode3/
```

Cada carpeta debe contener solamente los bloques asignados a ese DataNode. La distribución esperada para un archivo dividido en tres bloques y factor de replicación dos podría ser:

```
bloque_0 -> dn1, dn2  
bloque_1 -> dn2, dn3  
bloque_2 -> dn3, dn1
```

4.9 Replicación y tolerancia básica a fallos

La replicación consiste en almacenar más de una copia de un bloque. En DFS-Bloques, el factor inicial es dos, lo que significa que cada bloque debe quedar en dos DataNodes distintos. Esto permite que si un DataNode deja de responder, el sistema pueda intentar recuperar el bloque desde otra ubicación.

La tolerancia a fallos propuesta es básica. El sistema no garantiza disponibilidad total en todos los escenarios. Si todas las réplicas de un bloque se pierden, el archivo no puede reconstruirse. Sin embargo, sí debe cumplir con una propiedad mínima: si un DataNode falla y cada bloque aún tiene al menos una réplica disponible, el archivo debe poder recuperarse.

Durante la lectura, el cliente debe intentar descargar el bloque desde la primera ubicación disponible. Si ocurre timeout, error de conexión o error HTTP, el cliente debe registrar el evento y probar con la

siguiente réplica. Si ninguna réplica responde, debe reportar un error controlado como `BLOCK_NOT_AVAILABLE`.

4.10 Middleware REST/HTTP

La comunicación entre componentes se define mediante REST sobre HTTP. REST permite representar recursos mediante endpoints y usar métodos HTTP como GET, POST y DELETE. En este proyecto, REST es adecuado porque facilita la implementación, la depuración y la documentación automática mediante herramientas como Swagger cuando se usa FastAPI.

Los mensajes de control se envían en JSON. Los bloques se transfieren como contenido binario o mediante multipart/form-data. Así se separan dos tipos de tráfico:

- Tráfico de control: autenticación, metadatos, asignaciones, confirmaciones y listados.
- Tráfico de datos: envío y descarga de bloques.

5. Descripción del servicio y problema abordado

5.1 Servicio propuesto

DFS-Bloques es un servicio de almacenamiento distribuido minimalista. Permite que un usuario suba, descargue, liste y elimine archivos sin conocer manualmente en qué nodo se almacena cada bloque. El usuario interactúa con el sistema mediante comandos de alto nivel.

Las operaciones principales son:

```
python client/cli.py login --user santiago --password 1234
python client/cli.py put ./archivo.pdf --name archivo.pdf --block-size-mb 1
python client/cli.py ls
python client/cli.py get archivo.pdf --output ./downloads/archivo.pdf
python client/cli.py rm archivo.pdf
python client/cli.py mkdir documentos
python client/cli.py rmdir documentos
```

El servicio opera de la siguiente manera general:

1. El cliente se autentica.
2. El cliente divide el archivo en bloques.
3. El NameNode asigna DataNodes para cada bloque.
4. El cliente envía cada bloque a los DataNodes asignados.
5. Los DataNodes almacenan los bloques.
6. El NameNode registra los metadatos.
7. Para descargar, el cliente consulta los metadatos y reconstruye el archivo.

5.2 Problema abordado

El almacenamiento de archivos grandes en un único servidor crea limitaciones de disponibilidad, escalabilidad y tolerancia a fallos. Si el servidor falla, el archivo deja de estar disponible. Si el volumen de datos crece, una sola máquina puede no ser suficiente. Además, una arquitectura centralizada para datos impide demostrar distribución real.

DFS-Bloques aborda este problema dividiendo cada archivo en bloques y almacenando dichos bloques en varios DataNodes. La replicación permite que exista más de una copia de cada bloque, y el NameNode mantiene la tabla de ubicación necesaria para reconstruir los archivos. Esta estrategia permite demostrar cómo un sistema distribuido puede mantener acceso a datos aunque un nodo de almacenamiento falle parcialmente.

6. Alcance del proyecto

6.1 Alcance incluido

El sistema contempla:

- Cliente CLI/API para operaciones básicas.
- NameNode central de metadatos.
- Tres DataNodes como mínimo.
- API REST para comunicación entre componentes.
- Autenticación básica de usuarios.
- Aislamiento lógico por usuario.
- Particionamiento de archivos en bloques.
- Replicación de cada bloque en dos DataNodes.
- Reconstrucción de archivos desde bloques ordenados.
- Manejo básico de caída de DataNode durante lectura.
- Ejecución local con Docker Compose.
- Ejecución final mediante Docker Compose local, con AWS Academy documentado como alternativa de despliegue.
- Documentación de arquitectura, endpoints, algoritmos, pruebas y evidencias.

6.2 Fuera de alcance

No se incluye en la versión base:

- Integración real con el sistema de archivos del sistema operativo mediante montaje.
- Compatibilidad POSIX.
- Actualización parcial de bloques ya almacenados.
- Escrituras concurrentes sobre el mismo archivo.
- Alta disponibilidad del NameNode.
- Elección automática de un nuevo NameNode.
- Consenso distribuido.
- Re-replicación automática completa en segundo plano, salvo como mejora futura.
- Cifrado extremo a extremo.
- Permisos avanzados por usuario, grupo y rol.
- Balanceo por capacidad real o latencia.
- Alta disponibilidad del NameNode y re-replicación automática avanzada en segundo plano.

7. Requerimientos funcionales y no funcionales

7.1 Requerimientos funcionales

ID	Requerimiento	Descripción
RF01	Autenticación	El sistema debe permitir iniciar sesión con usuario y contraseña.
RF02	Aislamiento por usuario	Cada usuario debe gestionar únicamente sus archivos y directorios.
RF03	Subida de archivo	El cliente debe permitir put <archivo>.
RF04	Particionamiento	El cliente debe dividir el archivo en bloques configurables.
RF05	Asignación	El NameNode debe asignar DataNodes activos por bloque.
RF06	Replicación	Cada bloque debe almacenarse en al menos dos DataNodes distintos.
RF07	Commit	El archivo solo debe marcarse como disponible si los bloques fueron confirmados.
RF08	Metadatos	El NameNode debe registrar archivo, bloques, orden y ubicaciones.
RF09	Descarga	El cliente debe permitir get <archivo>.
RF10	Reconstrucción	El cliente debe reconstruir el archivo en el orden correcto.
RF11	Listado	El cliente debe permitir ls.
RF12	Eliminación	El cliente debe permitir rm <archivo>.
RF13	Directorios	El cliente debe permitir mkdir y rmdir.
RF14	Registro de nodos	Los DataNodes deben registrarse o estar configurados en el NameNode.
RF15	Health checks	El sistema debe validar si los DataNodes están activos.
RF16	API documentada	Los endpoints deben poder consultarse y probarse mediante documentación.

7.2 Requerimientos no funcionales

ID	Atributo	Descripción
RNF01	Portabilidad	El sistema debe ejecutarse en Linux o Docker.

ID	Atributo	Descripción
RNF02	Configurabilidad	URLs, puertos, tamaño de bloque y factor de replicación deben parametrizarse.
RNF03	Disponibilidad mínima	El archivo debe poder recuperarse si al menos una réplica por bloque está disponible.
RNF04	Modularidad	Cliente, NameNode y DataNode deben ser componentes independientes.
RNF05	Observabilidad	Deben existir logs o salidas claras de operaciones y errores.
RNF06	Simplicidad	La implementación debe ser minimalista y entendible.
RNF07	Trazabilidad	Los metadatos deben permitir rastrear bloques y ubicaciones.
RNF08	Recuperación controlada	Los errores deben producir mensajes claros.
RNF09	Extensibilidad	La arquitectura debe permitir agregar mejoras futuras.

8. Arquitectura del sistema

8.1 Estilo arquitectónico

La arquitectura combina Cliente/Servidor con Maestro-Trabajador. El sistema no se plantea como P2P porque se requiere una autoridad central para administrar metadatos y simplificar la localización de bloques. El NameNode actúa como maestro y los DataNodes como trabajadores. El cliente se comunica con ambos: con el NameNode para control y metadatos; con los DataNodes para transferencia de bloques.

Esta decisión permite mantener una división clara de responsabilidades:

- El cliente ejecuta comandos y transfiere bloques.
- El NameNode decide y registra dónde se ubican los bloques.
- Los DataNodes almacenan datos físicos.

8.2 Diagrama de arquitectura general

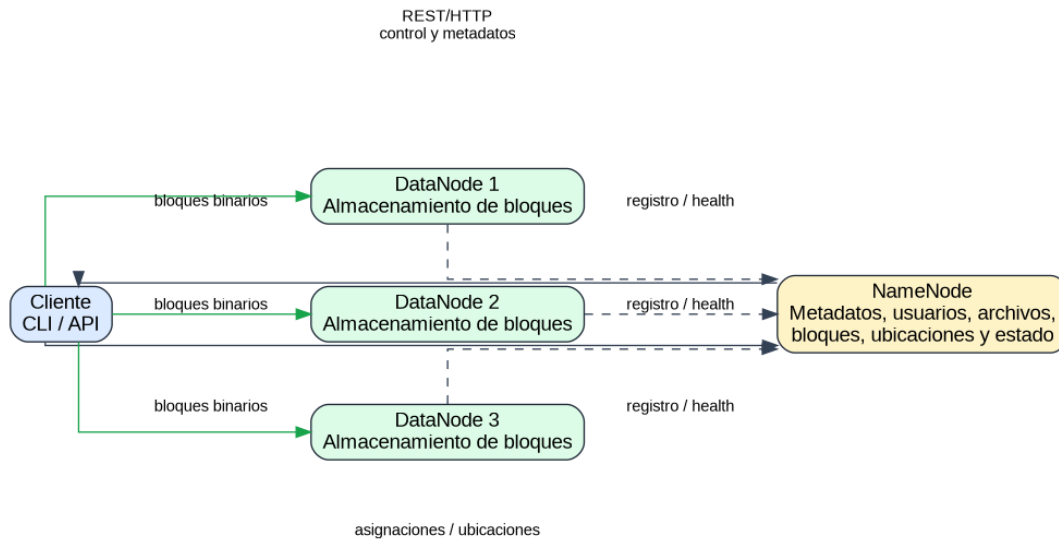


Figura 1. Arquitectura general propuesta del DFS por bloques.

Descripción: La figura muestra que el cliente consulta al NameNode para operaciones de control, mientras que la transferencia de bloques ocurre directamente entre el cliente y los DataNodes. Los DataNodes reportan estado o se registran ante el NameNode.

8.3 Componentes

Cliente CLI/API

Responsabilidades:

- Autenticar usuario.
- Leer archivos locales.
- Dividir archivos en bloques.
- Solicitar asignaciones al NameNode.
- Enviar bloques a DataNodes.
- Descargar bloques desde DataNodes.
- Reconstruir archivos.
- Ejecutar operaciones ls, rm, mkdir y rmdir.
- Mostrar errores controlados.

NameNode

Responsabilidades:

- Gestionar usuarios y autenticación básica.
- Registrar y consultar DataNodes.
- Mantener metadatos de archivos y bloques.
- Asignar DataNodes para cada bloque.

- Validar que exista factor mínimo de replicación.
- Entregar ubicaciones para descarga.
- Procesar eliminación lógica de archivos.
- Consultar estado de DataNodes.

DataNode

Responsabilidades:

- Exponer endpoint de salud.
- Recibir bloques.
- Guardar bloques en disco local.
- Entregar bloques al cliente.
- Eliminar bloques.
- Listar bloques para verificación.

8.4 Vista de despliegue local

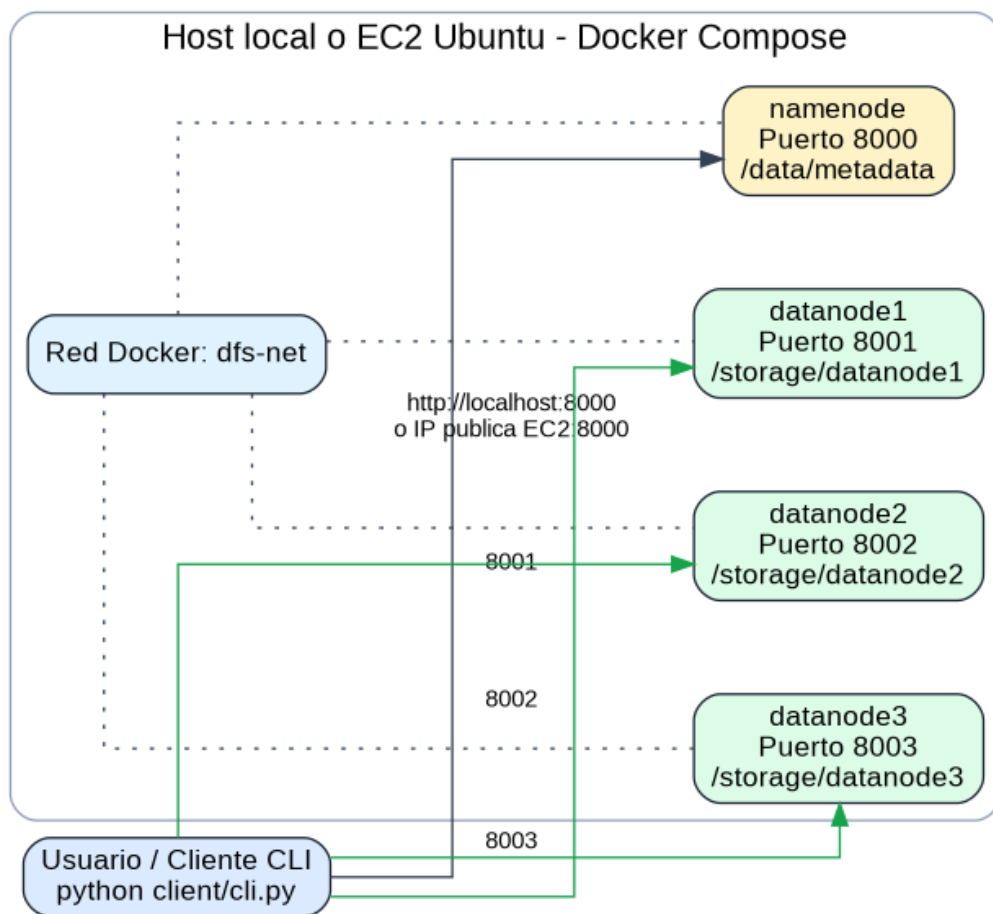


Figura 2. Vista de despliegue local o EC2 con Docker Compose.

Descripción: Para la demostración final se ejecuta el sistema con Docker Compose en una máquina local. La solución levanta cuatro contenedores independientes: un NameNode y tres DataNodes. Cada servicio expone su propia API REST en un puerto diferente y cada DataNode utiliza una ruta de almacenamiento separada, lo que permite evidenciar distribución de bloques entre nodos/contenedores.

8.5 Vista lógica de metadatos

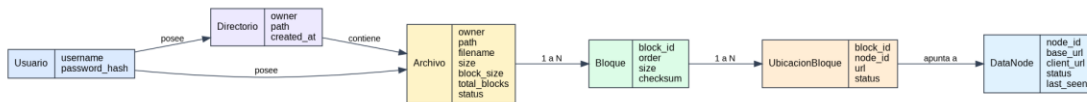


Figura 3. Modelo lógico de metadatos administrado por el NameNode.

Descripción: El modelo relaciona usuarios, directorios, archivos, bloques, ubicaciones y DataNodes. Esta relación permite reconstruir archivos y verificar la replicación.

8.6 Flujo de subida put

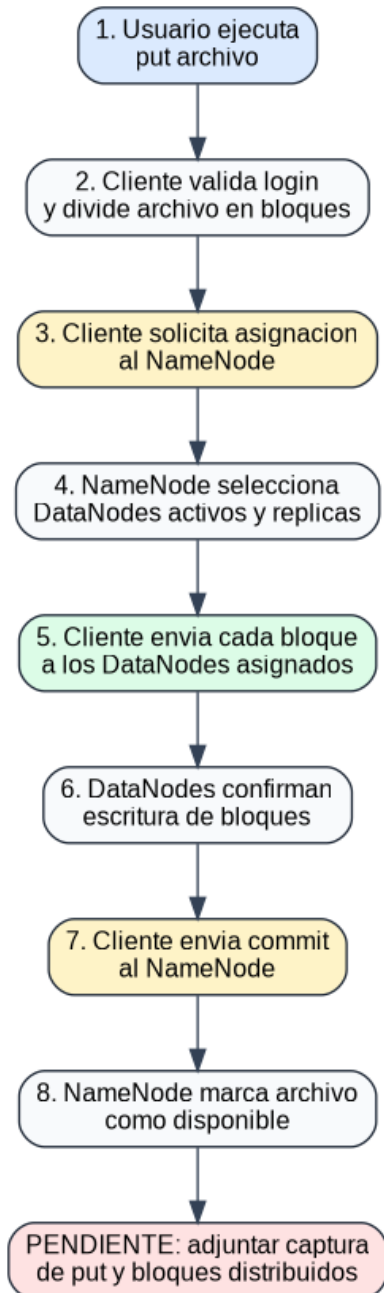


Figura 4. Flujo propuesto para la operación put.

En la operación put, el cliente divide el archivo, solicita al NameNode la asignación de réplicas, envía los bloques a los DataNodes y confirma el commit. El NameNode solo marca el archivo como disponible si se completó la escritura mínima requerida.

8.7 Flujo de descarga get

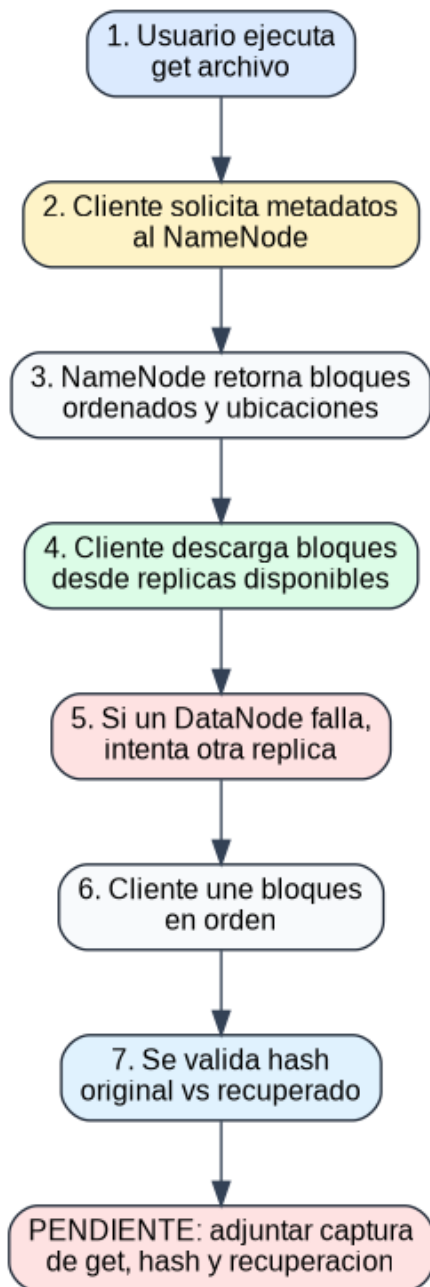


Figura 5. Flujo propuesto para la operación get.

En la operación get, el cliente consulta el NameNode para obtener la lista de bloques y ubicaciones. Luego descarga cada bloque desde una réplica disponible. Si una réplica falla, intenta con otra. Finalmente, reconstruye el archivo y se valida la integridad mediante comparación de hash.

8.8 Manejo arquitectónico de fallos

El fallo principal contemplado es la no disponibilidad de un DataNode. El sistema no debe depender de una única ubicación por bloque. Por eso, cada bloque se guarda en dos DataNodes diferentes. Si un DataNode no responde durante la lectura, el cliente intenta descargar el bloque desde la siguiente réplica registrada.

Durante la escritura, si no se logra almacenar un bloque en la cantidad mínima de réplicas, el archivo no debe confirmarse como disponible. Esto evita inconsistencias donde el NameNode indique que un archivo existe aunque sus bloques no estén completos.

9. Especificación de protocolos y APIs

9.1 Decisión de protocolo

Se utiliza REST sobre HTTP. Los metadatos se serializan en JSON y los bloques se transfieren como contenido binario o archivo multipart. Esta decisión facilita pruebas con Swagger, navegador, curl, Postman o Thunder Client.

9.1.1 Comunicación entre componentes

Cliente ↔ NameNode: el cliente usa la API REST del NameNode para autenticación, solicitud de asignación de bloques, commit, listado de archivos, consulta de ubicaciones, eliminación y gestión de directorios.

DataNode ↔ NameNode: cada DataNode se registra ante el NameNode al iniciar, reportando su identificador, URL de acceso y estado. Con esta información, el NameNode mantiene la tabla de nodos disponibles y puede asignar réplicas.

Cliente ↔ DataNodes: el cliente envía y descarga bloques directamente desde los DataNodes asignados por el NameNode. Esta transferencia corresponde al plano de datos del sistema.

DataNode ↔ DataNode: en esta versión no existe transferencia directa entre DataNodes. La replicación se coordina mediante metadatos del NameNode y se ejecuta desde el cliente, que envía cada bloque a los DataNodes asignados. Esta decisión se alinea con el flujo general del enunciado, donde el cliente transfiere bloques directamente a los DataNodes y el NameNode conserva archivo -> lista de bloques -> ubicación en DataNodes. Como mejora futura podría implementarse replicación DataNode -> DataNode en cascada, pero no es necesaria para la versión mínima funcional.

9.2 API del NameNode

Método	Endpoint	Propósito
GET	/health	Verificar que el NameNode está activo.
POST	/auth/login	Autenticar usuario.
POST	/datanodes/register	Registrar un DataNode.
GET	/datanodes	Consultar DataNodes registrados.
POST	/files/put-request	Solicitar asignación para subir archivo.
POST	/files/commit	Confirmar que los bloques fueron almacenados.

Método	Endpoint	Propósito
GET	/files	Listar archivos del usuario.
GET	/files/{filename}/blocks	Obtener bloques y ubicaciones para descarga.
DELETE	/files/{filename}	Eliminar archivo y metadatos.
POST	/directories	Crear directorio lógico.
DELETE	/directories/{path}	Eliminar directorio lógico.

Ejemplo de solicitud de asignación

```
{
  "filename": "documento.pdf",
  "size": 5242880,
  "block_size": 1048576,
  "total_blocks": 5
}
```

Ejemplo de respuesta de asignación

```
{
  "filename": "documento.pdf",
  "assignments": [
    {
      "block_id": "santiago_A9F12C_block_0",
      "order": 0,
      "replicas": [
        "http://datanode1:8001",
        "http://datanode2:8002"
      ]
    }
  ]
}
```

9.3 API del DataNode

Método	Endpoint	Propósito
GET	/health	Verificar estado del DataNode.
POST	/blocks/{block_id}	Guardar un bloque.
GET	/blocks/{block_id}	Descargar un bloque.
DELETE	/blocks/{block_id}	Eliminar un bloque.
GET	/blocks	Listar bloques almacenados.

9.4 Formato de error esperado

```
{
  "error": "BLOCK_NOT_AVAILABLE",
  "message": "No fue posible recuperar el bloque desde ninguna réplica disponible.",
  "details": {
    "block_id": "santiago_A9F12C_block_2",
    "attempted_nodes": ["dn1", "dn3"]
  }
}
```

10. Algoritmos implementados

10.1 Particionamiento

Entrada: ruta_archivo, block_size

Salida: lista_bloques

1. Abrir el archivo en modo binario.
2. Inicializar order = 0.
3. Leer fragmentos de tamaño block_size.
4. Generar block_id único por fragmento.
5. Asociar order, tamaño y contenido.
6. Repetir hasta finalizar archivo.
7. Retornar lista ordenada de bloques.

10.2 Identificación de bloques

Se propone identificar bloques combinando usuario, hash del archivo o de la operación y número de orden:

block_id = <owner>_<file_hash>_block_<order>

Ejemplo:

santiago_A9F12C_block_0
santiago_A9F12C_block_1
santiago_A9F12C_block_2

10.3 Distribución round-robin

Entrada: bloques, datanodes_activos, factor_replicacion

Salida: asignaciones

1. Validar que la cantidad de DataNodes activos sea mayor o igual al factor.
2. Para cada bloque i:
 - a. Seleccionar DataNode en posición $(i + r) \bmod N$.
 - b. Repetir hasta alcanzar factor_replicacion.
 - c. Evitar asignar dos réplicas al mismo DataNode.
3. Retornar bloque -> lista de DataNodes.

Ejemplo con tres DataNodes y factor dos:

Bloque 0 -> dn1, dn2
Bloque 1 -> dn2, dn3
Bloque 2 -> dn3, dn1
Bloque 3 -> dn1, dn2

10.4 Subida put

1. Cliente valida autenticación.
2. Cliente valida existencia del archivo local.
3. Cliente calcula tamaño y total de bloques.
4. Cliente solicita asignación al NameNode.
5. NameNode retorna block_id, order y réplicas por bloque.
6. Cliente envía cada bloque a los DataNodes asignados.
7. DataNodes responden confirmación.

8. Si un bloque no alcanza réplicas mínimas, se aborta.
9. Si todos los bloques se guardan, cliente envía commit.
10. NameNode registra metadatos finales.

10.5 Descarga get

1. Cliente valida autenticación.
2. Cliente solicita metadatos del archivo al NameNode.
3. NameNode retorna bloques ordenados y ubicaciones.
4. Para cada bloque:
 - a. Intentar descargar desde la primera réplica.
 - b. Si falla, intentar la siguiente.
 - c. Si ninguna responde, abortar.
5. Escribir los bloques en orden en archivo destino.
6. Validar hash si está disponible.
7. Reportar descarga exitosa.

10.6 Eliminación rm

1. Cliente solicita eliminación al NameNode.
2. NameNode valida usuario y existencia del archivo.
3. NameNode obtiene bloques y ubicaciones.
4. Se solicita eliminación a los DataNodes correspondientes.
5. DataNodes eliminan bloques físicos.
6. NameNode elimina metadatos.
7. Sistema retorna confirmación.

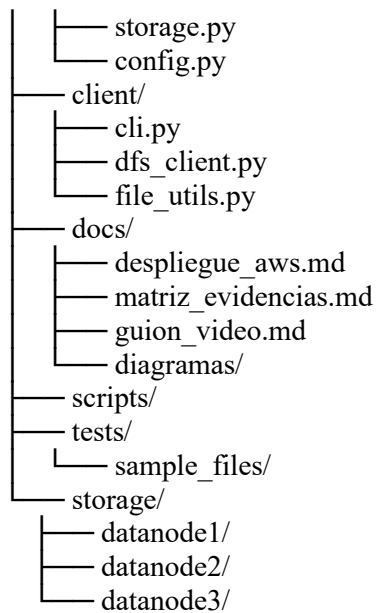
10.7 Tolerancia a fallos en lectura

1. Recibir block_id y lista de réplicas.
2. Intentar GET /blocks/{block_id} en la primera réplica.
3. Si responde correctamente, usar ese bloque.
4. Si ocurre timeout o error HTTP, registrar fallo.
5. Intentar con la siguiente réplica.
6. Si ninguna réplica responde, retornar BLOCK_NOT_AVAILABLE.

11. Implementación desarrollada y estructura del repositorio

La implementación se organiza en componentes separados. La estructura base del repositorio es:

```
dfs-bloques/
├── README.md
├── docker-compose.yml
├── Dockerfile
├── requirements.txt
├── .env.example
├── namenode/
│   ├── main.py
│   ├── auth.py
│   ├── metadata_store.py
│   ├── block_allocator.py
│   └── models.py
├── datanode/
│   └── main.py
```



11.1 NameNode

El NameNode concentra la lógica de metadatos. Debe exponer endpoints para autenticación, registro de DataNodes, asignación de bloques, confirmación de subida, consulta de archivos, consulta de bloques y eliminación.

11.2 DataNode

El DataNode debe ser un servicio independiente que recibe su configuración mediante variables de entorno. Cada instancia debe conocer su NODE_ID, puerto y ruta de almacenamiento. Su lógica principal consiste en guardar y entregar bloques.

11.3 Cliente CLI

El cliente CLI es la herramienta usada por el usuario. Debe comunicarse con el NameNode para control y con DataNodes para transferencia. Debe leer variables de configuración para localizar los servicios.

11.4 Documentación del repositorio

The screenshot shows a GitHub repository page for 'Proyecto-Sistemas-Distribuidos' by user 'josuepinop'. The repository is public and has 10 commits. The main branch is 'main'. The repository contains several files and folders: 'client', 'common', 'datanode', 'namenode', 'scripts', 'tests', 'Dockerfile', 'README_DFS_Bloques.md', 'docker-compose.yml', and 'requirements.txt'. The 'README' file is highlighted. The right sidebar shows the 'About' section with no description, website, or topics provided. It also shows 'Releases' and 'Packages' sections, both with no published items. The 'Contributors' section lists two contributors: 'santiagorepota14' and 'josuepinop'.

Project-Sistemas-Distribuidos

main 1 Branch 0 Tags

Go to file

Add file

Code

santiagorepota14 Delete README.md 1 minute ago 10 Commits

client	Add files via upload	13 minutes ago
common	Add files via upload	6 minutes ago
datanode	Add files via upload	6 minutes ago
namenode	Add files via upload	6 minutes ago
scripts	Add files via upload	13 minutes ago
tests	Add files via upload	12 minutes ago
Dockerfile	Add files via upload	5 minutes ago
README_DFS_Bloques.md	Add files via upload	1 minute ago
docker-compose.yml	Add files via upload	5 minutes ago
requirements.txt	Add files via upload	5 minutes ago

README

About

No description, website, or topics provided.

Activity

0 stars

0 watching

0 forks

Releases

No releases published

Create a new release

Packages

No packages published

Publish your first package

Contributors 2

santiagorepota14 Santiago Restre...

josuepinop

El repositorio incluye:

- README con instalación, ejecución y pruebas.
- .env.example con variables locales y notas para AWS.
- Guía de despliegue en AWS.
- Matriz de evidencias.
- Guion del video.
- Diagramas de arquitectura.

README

Si Docker está funcionando, debe aparecer una tabla de contenedores, aunque esté vacía.

Configuración inicial

Clonar el repositorio:

```
git clone https://github.com/josuepinop/Proyecto-Sistemas-Distribuidos-.git
cd Proyecto-Sistemas-Distribuidos-
```

Copiar archivo de variables:

```
copy .env.example .env
```

Ejecución con Docker Compose

Levantar el sistema completo:

```
docker compose up --build
```

Esto inicia:

```
dfs-namenode
dfs-datanode1
dfs-datanode2
dfs-datanode3
```

En otra terminal, verificar contenedores activos:

```
docker ps
```

Preparar entorno del cliente

En otra terminal de PowerShell:

12. Ambiente técnico y configuración

12.1 Tecnologías

Elemento	Tecnología	Justificación
Lenguaje	Python 3.11+	Sencillo para APIs, CLI y archivos.

Elemento	Tecnología	Justificación
API	FastAPI	Documentación automática y rapidez de desarrollo.
Servidor	Uvicorn	Servidor ASGI ligero.
Cliente CLI	Python con argparse o Typer	Permite comandos claros.
Persistencia de metadatos	JSON o SQLite	Suficiente para una versión académica.
Almacenamiento de bloques	Sistema de archivos local	Cada DataNode guarda bloques en carpeta propia.
Contenedores	Docker y Docker Compose	Permite simular nodos independientes.
Comunicación	REST/HTTP	Simple, documentable y fácil de probar.

12.2 Variables de configuración

Variable	Componente	Descripción	Ejemplo local
NAMENODE_URL	Cliente/DataNode	URL base del NameNode.	http://localhost:8000
NODE_ID	DataNode	Identificador del DataNode.	dn1
NODE_PORT	DataNode	Puerto del DataNode.	8001
STORAGE_PATH	DataNode	Carpeta de bloques.	/data/blocks
BLOCK_SIZE_MB	Cliente/NameNode	Tamaño de bloque por defecto.	64
REPLICATION_FACTOR	NameNode	Réplicas por bloque.	2
METADATA_DB	NameNode	Ruta de metadatos.	/data/metadata.db
REQUEST_TIMEOUT	Cliente/NameNode	Timeout de peticiones.	5

12.3 Ejecución local prevista

```
cp .env.example .env
docker compose up --build
```

En otra terminal:

```
python -m venv .venv
.venv\Scripts\Activate.ps1
pip install -r requirements.txt
```

```
python client/cli.py health
python client/cli.py datanodes
python client/cli.py login --user santiago --password 1234
python scripts/create_sample_file.py --output tests/sample_files/demo.bin --size-mb 3
python client/cli.py put tests/sample_files/demo.bin --name demo.bin --block-size-mb 1
python client/cli.py ls
python client/cli.py get demo.bin --output downloads/demo_recuperado.bin
```

La ejecución local fue validada preliminarmente con Docker Compose. Se verificó que el NameNode y los tres DataNodes levantarán correctamente, que los DataNodes se registraran ante el NameNode, que el cliente pudiera autenticarse, subir archivos, listar, descargar, comparar hash y recuperar un archivo aun con un DataNode detenido. Las capturas finales se deben ubicar en la sección de resultados y análisis.

13. Entorno de ejecución final con Docker Compose

De acuerdo con la aclaración realizada con el profesor, la demostración final del DFS se realizará mediante Docker Compose en entorno local. Este esquema es válido para el alcance del proyecto porque el enunciado permite ejecutar cada nodo en contenedores Docker y la rúbrica acepta la ejecución del DFS en al menos tres nodos distribuidos o contenedores. Por tanto, AWS Academy se mantiene como alternativa documentada, no como evidencia principal de la entrega.

El entorno final levanta cuatro contenedores independientes: un NameNode y tres DataNodes. El NameNode expone la API central de metadatos en el puerto 8000. Los DataNodes exponen APIs independientes en los puertos 8001, 8002 y 8003, y cada uno mantiene su propio almacenamiento de bloques.

13.1 Esquema de ejecución final

Máquina local con Docker Compose

- NameNode -> contenedor dfs-namenode -> puerto 8000
- DataNode 1 -> contenedor dfs-datanode1 -> puerto 8001
- DataNode 2 -> contenedor dfs-datanode2 -> puerto 8002
- DataNode 3 -> contenedor dfs-datanode3 -> puerto 8003

```
Windows PowerShell
Copyright (C) Microsoft Corporation. Todos los derechos reservados.

Instale la versión más reciente de PowerShell para obtener nuevas características y mejoras. https://aka.ms/PSWindows

PS C:\Users\santy\Desktop\SISTEMAS\SEXTO SEMESTRE\Nube y sistemas distribuidos\PROYECTO 1\dfs-bloques\dfs-bloques> docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS                               NAMES
63b8f8d39dff   dfs-bloques-datanode1  "uvicorn datanode.ma..." About a minute ago Up About a minute 0.0.0.0:8001->8001/tcp, [::]:8001->8001/tcp  dfs-datanode1
5626a5994ff03   dfs-bloques-datanode3  "uvicorn datanode.ma..." About a minute ago Up About a minute 0.0.0.0:8003->8003/tcp, [::]:8003->8003/tcp  dfs-datanode3
2d3c737efaad   dfs-bloques-datanode2  "uvicorn datanode.ma..." About a minute ago Up About a minute 0.0.0.0:8002->8002/tcp, [::]:8002->8002/tcp  dfs-datanode2
922ade38552e   dfs-bloques-namenode   "uvicorn namenode.ma..." About a minute ago Up About a minute 0.0.0.0:8000->8000/tcp, [::]:8000->8000/tcp  dfs-namenode
PS C:\Users\santy\Desktop\SISTEMAS\SEXTO SEMESTRE\Nube y sistemas distribuidos\PROYECTO 1\dfs-bloques\dfs-bloques> |
```

13.2 Variables usadas en local

NAMENODE_URL=http://localhost:8000
DATANODE1_CLIENT_URL=http://localhost:8001
DATANODE2_CLIENT_URL=http://localhost:8002
DATANODE3_CLIENT_URL=http://localhost:8003
BLOCK_SIZE_MB=64
REPLICATION_FACTOR=2

13.3 Comandos principales de ejecución

cp .env.example .env
docker compose up --build

En otra terminal:

python -m venv .venv
.venv\Scripts\Activate.ps1


```
pip install -r requirements.txt
python client/cli.py health
python client/cli.py datanodes
python client/cli.py login --user santiago --password 1234
python scripts/create_sample_file.py --output tests/sample_files/demo.bin --size-mb 3
python client/cli.py put tests/sample_files/demo.bin --name demo.bin --block-size-mb 1
python client/cli.py ls
python client/cli.py get demo.bin --output downloads/demo_recuperado.bin
```

13.4 AWS Academy como alternativa documentada

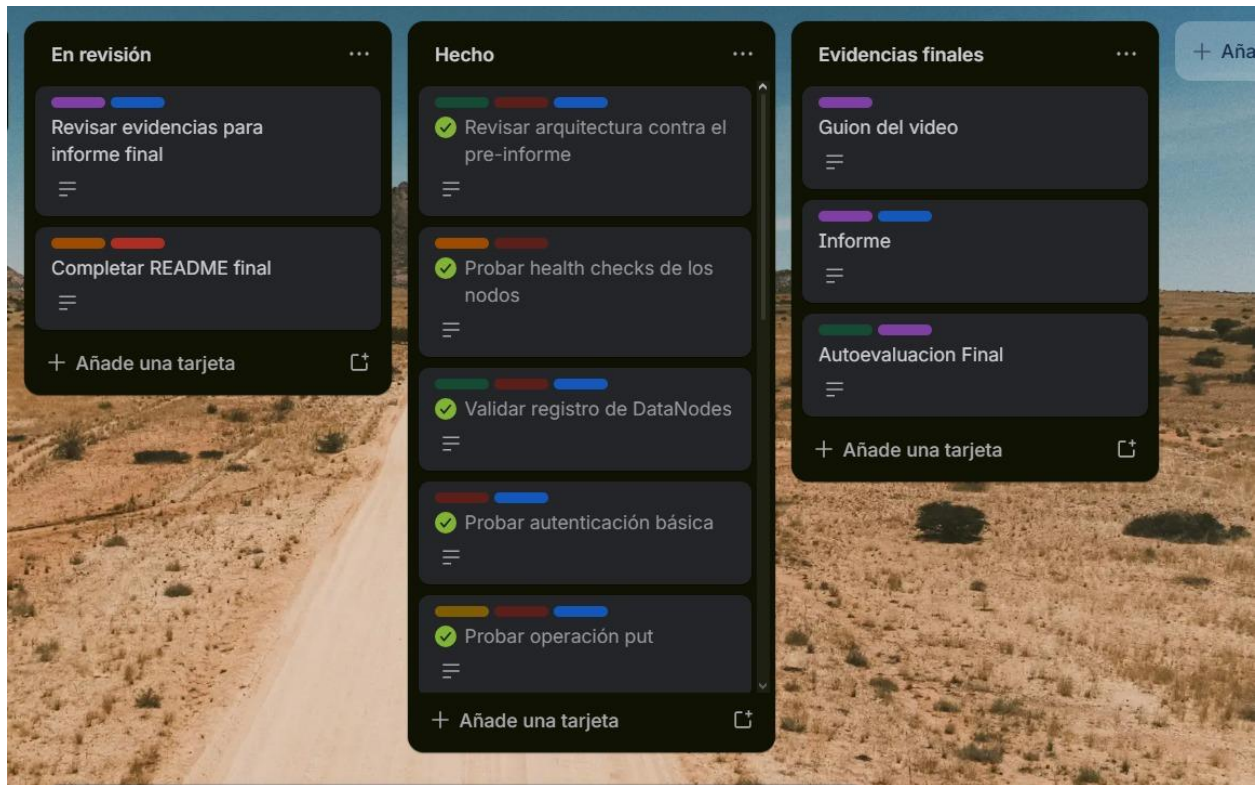
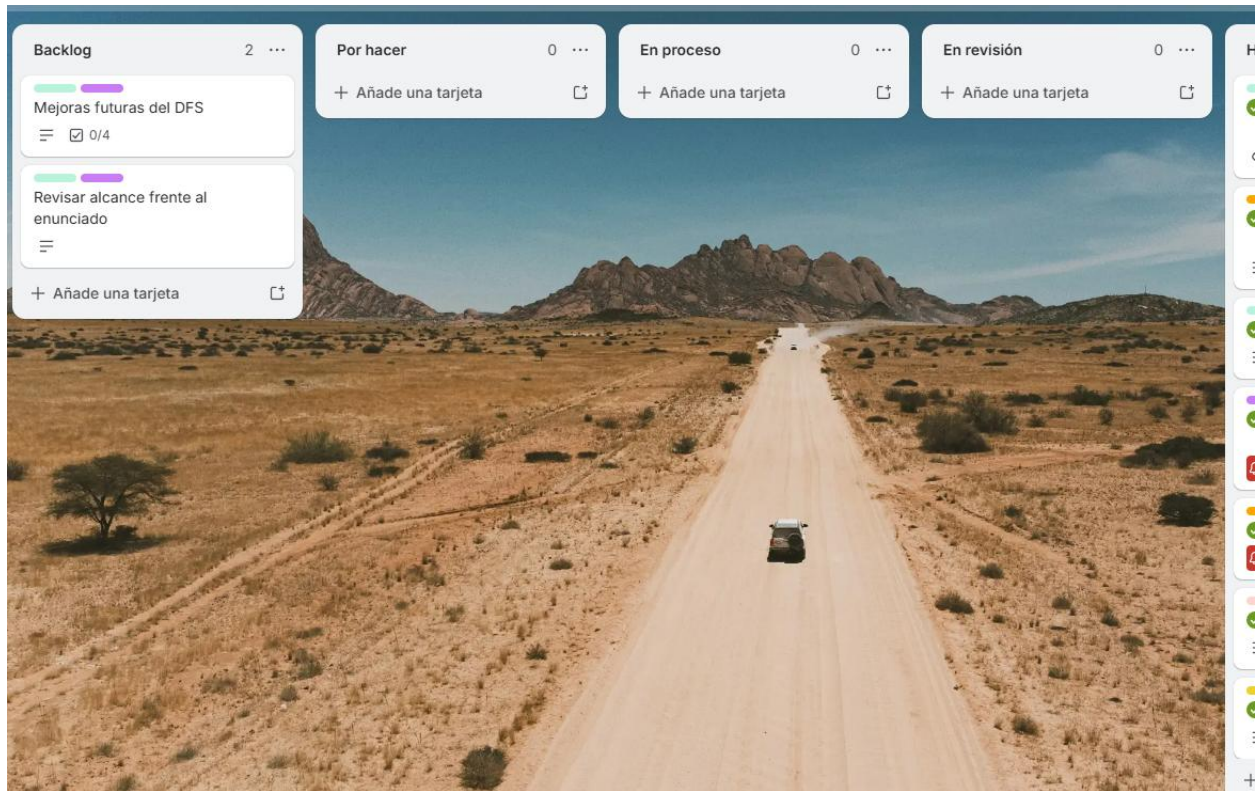
AWS Academy queda documentado como alternativa de despliegue, pero no será el entorno principal de evidencia para esta entrega. Si posteriormente se decide usar AWS Academy, el mismo proyecto puede ejecutarse en una EC2 con Docker Compose o distribuirse en varias máquinas ajustando las URLs/IPs de los DataNodes. Para la entrega actual, la demostración se centra en Docker local con un NameNode y tres DataNodes como contenedores independientes, lo cual permite evidenciar particionamiento, replicación y comunicación entre nodos/contenedores.

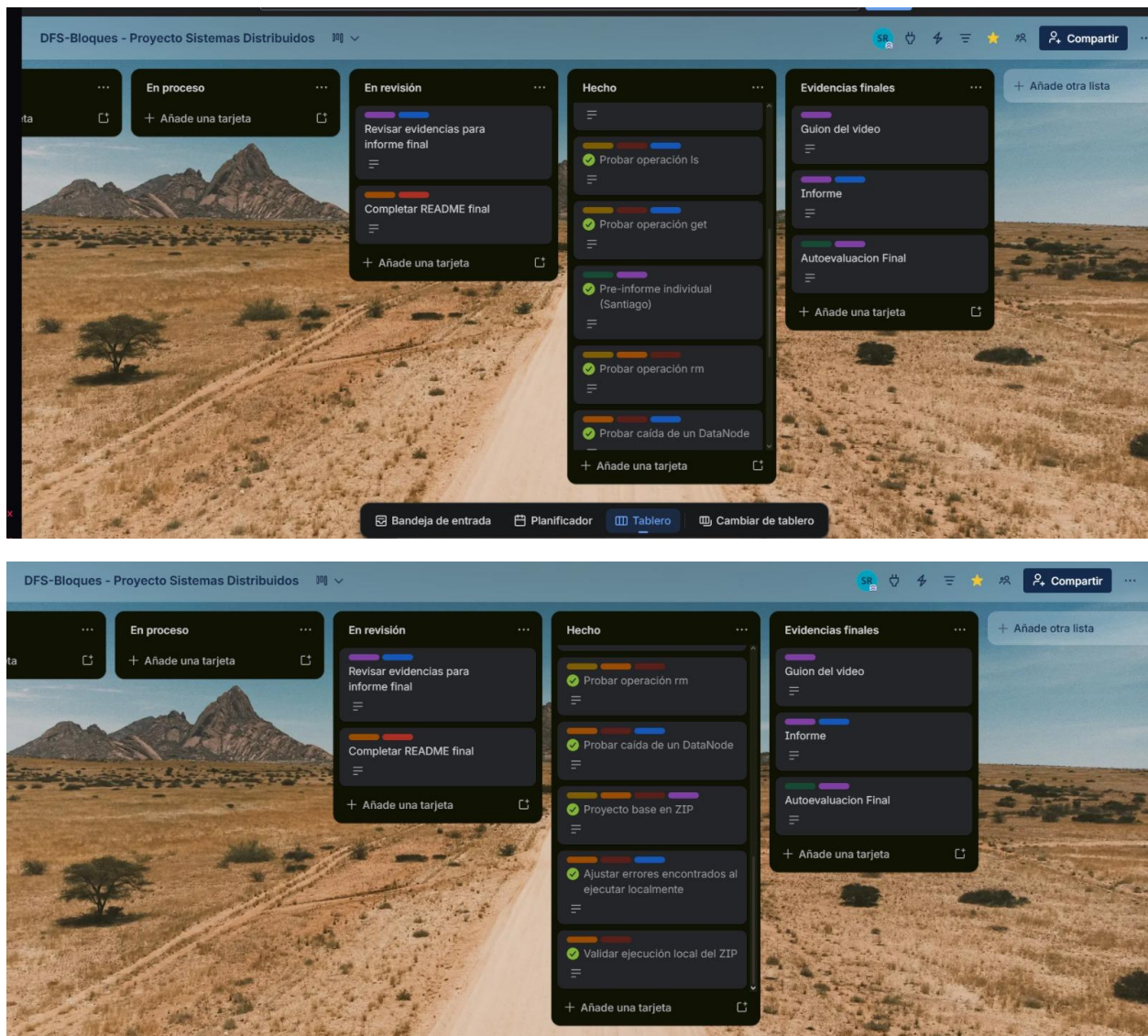
14. Gestión del proyecto con Trello

Para evidenciar la gestión del trabajo en grupo se creó un tablero en Trello con listas de avance, etiquetas, responsables y tarjetas por actividad. La estructura propuesta es:

- Backlog
- Por hacer
- En proceso
- En revisión
- Hecho
- Evidencias finales

Las tarjetas principales incluyen validación del ZIP, health checks, registro de DataNodes, autenticación, pruebas de put, get, ls, rm, mkdir, rmdir, prueba de caída de DataNode, revisión de evidencias, README final, guion de video y autoevaluación final.





15. Plan de pruebas

Prueba	Objetivo	Evidencia esperada
Ejecución local	Validar que los contenedores levanten.	Captura de docker compose up.
Health checks	Confirmar servicios activos.	Respuestas /health.
Registro DataNodes	Ver que el NameNode conoce dn1, dn2 y dn3.	Listado de DataNodes.
Login	Probar autenticación correcta e incorrecta.	Mensaje de éxito/error.

Prueba	Objetivo	Evidencia esperada
put	Subir archivo y dividir en bloques.	Consola y bloques físicos.
Distribución	Confirmar bloques en varios DataNodes.	Carpetas de DataNodes.
Replicación	Confirmar dos copias por bloque.	Metadatos y carpetas.
ls	Listar archivos del usuario.	Salida del cliente.
get	Reconstruir archivo.	Archivo recuperado.
Hash	Validar integridad.	SHA-256 idéntico.
Caída de DataNode	Probar tolerancia básica.	get exitoso con un nodo caído.
rm	Eliminar archivo y bloques.	Metadatos y carpetas limpias.
mkdir/rmdir	Gestionar directorios.	Salida de CLI.
AWS opcional	Documentar alternativa de despliegue.	Guía o sección de alternativa AWS.
Logs y errores controlados	Evidenciar manejo de fallas, especialmente DataNode caído.	Salida del cliente o logs de Docker mostrando error controlado y recuperación.
Repositorio	Evidenciar código fuente documentado.	Captura de GitHub y README.
Trello final	Evidenciar gestión, asignación y seguimiento de tareas.	Captura final del tablero con responsables, etiquetas y avance.
Video y autoevaluación	Cumplir entregables obligatorios y evitar penalidades.	Archivo/enlace de video y plantilla de autoevaluación diligenciada.

16. Resultados y análisis de pruebas

16.1 Ejecución del clúster Docker

```
Windows PowerShell
Copyright (C) Microsoft Corporation. Todos los derechos reservados.

Instale la versión más reciente de PowerShell para obtener nuevas características y mejoras. https://aka.ms/PSWindows

PS C:\Users\santy\Desktop\SISTEMAS\SEXTO SEMESTRE\Nube y sistemas distribuidos\PROYECTO 1\dfs-bloques\dfs-bloques> docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS                               NAMES
63b8f8d39dff   dfs-bloques-datanode1   "uvicorn datanode.ma..." About a minute ago    Up About a minute    0.0.0.0:8001->8001/tcp, [::]:8001->8001/tcp   dfs-d
atanode1
5626a5994f03   dfs-bloques-datanode3   "uvicorn datanode.ma..." About a minute ago    Up About a minute    0.0.0.0:8003->8003/tcp, [::]:8003->8003/tcp   dfs-d
atanode3
2d3c737efaad   dfs-bloques-datanode2   "uvicorn datanode.ma..." About a minute ago    Up About a minute    0.0.0.0:8002->8002/tcp, [::]:8002->8002/tcp   dfs-d
atanode2
922ade38552e   dfs-bloques-namenode     "uvicorn namenode.ma..." About a minute ago    Up About a minute    0.0.0.0:8000->8000/tcp, [::]:8000->8000/tcp   dfs-n
amenode
PS C:\Users\santy\Desktop\SISTEMAS\SEXTO SEMESTRE\Nube y sistemas distribuidos\PROYECTO 1\dfs-bloques\dfs-bloques> |
```

Análisis esperado: esta evidencia demuestra que el DFS se ejecuta en al menos tres DataNodes como contenedores independientes, cumpliendo el criterio de entorno distribuido o contenerizado.

16.2 APIs y health checks

localhost:8000/docs/

DFS-Bloques NameNode 1.0.0 OAS 3.1

/openapi.json

Servidor central de metadatos para un DFS académico por bloques.

default

Schemas

CommitRequest > Expand all object

CommittedBlock > Expand all object

DataNodeRegisterRequest > Expand all object

DirectoryRequest > Expand all object

HTTPValidationError > Expand all object

LoginRequest > Expand all object

DFS-Bloques DataNode dn1 1.0.0 OAS 3.1

/openapi.json

Nodo de almacenamiento físico de bloques para DFS-Bloques.

default

GET /health Health

POST /blocks/{block_id} Put Block

GET /blocks/{block_id} Get Block

DELETE /blocks/{block_id} Delete Block

GET /blocks List Blocks

Schemas

HTTPValidationError > Expand all object

ValidationError > Expand all object

DFS-Bloques DataNode dn2 1.0.0 OAS 3.1

/openapi.json

Nodo de almacenamiento fisico de bloques para DFS-Bloques.

default

GET	/health	Health	▼
POST	/blocks/{block_id}	Put Block	▼
GET	/blocks/{block_id}	Get Block	▼
DELETE	/blocks/{block_id}	Delete Block	▼
GET	/blocks	List Blocks	▼

Schemas

HTTPValidationError > Expand all object

ValidationError > Expand all object

DFS-Bloques DataNode dn3 1.0.0 OAS 3.1

/openapi.json

Nodo de almacenamiento fisico de bloques para DFS-Bloques.

default

GET	/health	Health	▼
POST	/blocks/{block_id}	Put Block	▼
GET	/blocks/{block_id}	Get Block	▼
DELETE	/blocks/{block_id}	Delete Block	▼
GET	/blocks	List Blocks	▼

Schemas

HTTPValidationError > Expand all object

ValidationError > Expand all object

```
PS C:\Users\santy\Desktop\SISTEMAS\SEXTO SEMESTRE\Nube y sistemas distribuidos\PROYECTO 1\dfs-bloques\dfs-bloques> .\venv\Scripts\Activate.ps1
(.venv) PS C:\Users\santy\Desktop\SISTEMAS\SEXTO SEMESTRE\Nube y sistemas distribuidos\PROYECTO 1\dfs-bloques\dfs-bloques> python client/cli.py health
{
  "service": "namenode",
  "status": "ok",
  "replication_factor": 2,
  "metadata_db": "/data/metadata.json"
}
(.venv) PS C:\Users\santy\Desktop\SISTEMAS\SEXTO SEMESTRE\Nube y sistemas distribuidos\PROYECTO 1\dfs-bloques\dfs-bloques> |
```

16.3 Registro de DataNodes

```
(.venv) PS C:\Users\santy\Desktop\SISTEMAS\SEXTO SEMESTRE\Nube y sistemas distribuidos\PROYECTO 1\dfs-bloques\dfs-bloques> python client/cli.py datanodes
{
  "count": 3,
  "datanodes": [
    {
      "node_id": "dn1",
      "base_url": "http://datanode1:8001",
      "client_url": "http://localhost:8001",
      "status": "active",
      "last_seen": 1779470631.4779483
    },
    {
      "node_id": "dn2",
      "base_url": "http://datanode2:8002",
      "client_url": "http://localhost:8002",
      "status": "active",
      "last_seen": 1779470631.496017
    },
    {
      "node_id": "dn3",
      "base_url": "http://datanode3:8003",
      "client_url": "http://localhost:8003",
      "status": "active",
      "last_seen": 1779470631.5108862
    }
  ]
}
```

Análisis esperado: el NameNode debe conocer los DataNodes disponibles y sus URLs. Esto evidencia la comunicación DataNode -> NameNode y la existencia de metadatos de nodos.

16.4 Autenticación

```
(.venv) PS C:\Users\santy\Desktop\SISTEMAS\SEXTO SEMESTRE\Nube y sistemas distribuidos\PROYECTO 1\dfs-bloques\dfs-bloques> python client/cli.py login --user
santiago --password 1234
{
  "token": "p8vubo4WZebRRlpqHytIpqFkYvpZ9QoL-0_T_LSDQU",
  "username": "santiago",
  "message": "Login exitoso."
}
```

```
(.venv) PS C:\Users\santy\Desktop\SISTEMAS\SEXTO SEMESTRE\Nube y sistemas distribuidos\PROYECTO 1\dfs-bloques\dfs-bloques> python client/cli.py login --user
santiago --password mala
ERROR: HTTP 401: Usuario o contraseña inválidos.
```

16.5 Prueba put: particionamiento, distribución y replicación

```
Directorio: C:\Users\santy\Desktop\SISTEMAS\SEXTO SEMESTRE\Nube y sistemas distribuidos\PROYECTO 1\dfs-bloques\dfs-bloques\tests

Mode                LastWriteTime         Length Name
----                -
d-----            3/05/2026   1:21 p. m.         sample_files

(.venv) PS C:\Users\santy\Desktop\SISTEMAS\SEXTO SEMESTRE\Nube y sistemas distribuidos\PROYECTO 1\dfs-bloques\dfs-bloques> New-Item -ItemType Directory -For
ce -Path "downloads"

Directorio: C:\Users\santy\Desktop\SISTEMAS\SEXTO SEMESTRE\Nube y sistemas distribuidos\PROYECTO 1\dfs-bloques\dfs-bloques

Mode                LastWriteTime         Length Name
----                -
d-----            22/05/2026   12:26 p. m.         downloads

(.venv) PS C:\Users\santy\Desktop\SISTEMAS\SEXTO SEMESTRE\Nube y sistemas distribuidos\PROYECTO 1\dfs-bloques\dfs-bloques> python scripts/create_sample_file
.py --output tests/sample_files/demo.bin --size-mb 3
Archivo creado: tests/sample_files/demo.bin (3145728 bytes)
(.venv) PS C:\Users\santy\Desktop\SISTEMAS\SEXTO SEMESTRE\Nube y sistemas distribuidos\PROYECTO 1\dfs-bloques\dfs-bloques> Get-ChildItem tests/sample_files\
demo.bin

Directorio: C:\Users\santy\Desktop\SISTEMAS\SEXTO SEMESTRE\Nube y sistemas distribuidos\PROYECTO 1\dfs-bloques\dfs-bloques\tests\sample_files

Mode                LastWriteTime         Length Name
----                -
-a-----            22/05/2026   12:26 p. m.      3145728 demo.bin
```

```
(.venv) PS C:\Users\santy\Desktop\SISTEMAS\SEXTO SEMESTRE\Nube y sistemas distribuidos\PROYECTO 1\dfs-bloques\dfs-bloques> python client/cli.py put tests/sample_files/demo.bin --name demo.bin --block-size-mb 1
{
  "message": "Archivo confirmado correctamente.",
  "file": {
    "filename": "demo.bin",
    "owner": "santiago",
    "size": 3145728,
    "block_size": 1048576,
    "total_blocks": 3,
    "file_checksum": "22bade239b0df05e2683496736237bcdcfceaab43a15e56c8fb9b215d70a03c5e",
    "replication_factor": 2,
    "status": "available",
    "created_at": 1779470863.4190936,
    "blocks": [
      {
        "block_id": "santiago_aaba0a6b0a2d_block_0",
        "order": 0,
        "size": 1048576,
        "checksum": "ff6b855f63853f50598c6c30e3befec822cb7c40bf4a87a366815c5bdf02f8f1",
        "locations": [
          {
            "node_id": "dn1",
            "client_url": "http://localhost:8001",
            "base_url": "http://datanode1:8001"
          },
          {
            "node_id": "dn2",
            "client_url": "http://localhost:8002",
            "base_url": "http://datanode2:8002"
          }
        ]
      },
      {
        "block_id": "santiago_aaba0a6b0a2d_block_1",
        "order": 1,
        "size": 1048576,
        "checksum": "2c713116fe65f78bf374019067af63f94d8636d59ac10901a96426ede149f94f",

```

```

      {
        "block_id": "santiago_aaba0a6b0a2d_block_1",
        "order": 1,
        "size": 1048576,
        "checksum": "2c713116fe65f78bf374019067af63f94d8636d59ac10901a96426ede149f94f",
        "locations": [
          {
            "node_id": "dn2",
            "client_url": "http://localhost:8002",
            "base_url": "http://datanode2:8002"
          },
          {
            "node_id": "dn3",
            "client_url": "http://localhost:8003",
            "base_url": "http://datanode3:8003"
          }
        ]
      },
      {
        "block_id": "santiago_aaba0a6b0a2d_block_2",
        "order": 2,
        "size": 1048576,
        "checksum": "2c90e7cb6c38ec89e0a7b8d22ebb1a320964ecc046e3c848b1bfc52178382575",
        "locations": [
          {
            "node_id": "dn3",
            "client_url": "http://localhost:8003",
            "base_url": "http://datanode3:8003"
          },
          {
            "node_id": "dn1",
            "client_url": "http://localhost:8001",
            "base_url": "http://datanode1:8001"
          }
        ]
      }
    ],
    "local_file_checksum": "22bade239b0df05e2683496736237bcdcfceaab43a15e56c8fb9b215d70a03c5e"
  }
}

```

```
(.venv) PS C:\Users\santy\Desktop\SISTEMAS\SEXTO SEMESTRE\Nube y sistemas distribuidos\PROYECTO 1\dfs-bloques\dfs-bloques> Invoke-RestMethod http://localhost:8001/blocks | ConvertTo-Json -Depth 10
{
  "node_id": "dn1",
  "count": 2,
  "blocks": [
    {
      "block_id": "santiago_aaba0a6b0a2d_block_0",
      "stored_name": "santiago_aaba0a6b0a2d_block_0",
      "size": 1048576,
      "checksum": "ff6b855f63853f50598c6c30e3befec822cb7c40bf4a87a366815c5bdf02f8f1"
    },
    {
      "block_id": "santiago_aaba0a6b0a2d_block_2",
      "stored_name": "santiago_aaba0a6b0a2d_block_2",
      "size": 1048576,
      "checksum": "2c90e7cb6c38ec89e0a7b8d22ebb1a320964ecc046e3c848b1bfc52178382575"
    }
  ]
}
(.venv) PS C:\Users\santy\Desktop\SISTEMAS\SEXTO SEMESTRE\Nube y sistemas distribuidos\PROYECTO 1\dfs-bloques\dfs-bloques> |
```



```
(.venv) PS C:\Users\santy\Desktop\SISTEMAS\SEXTO SEMESTRE\Nube y sistemas distribuidos\PROYECTO 1\dfs-bloques\dfs-bloques> Invoke-RestMethod http://localhost:8002/blocks | ConvertTo-Json -Depth 10
{
  "node_id": "dn2",
  "count": 2,
  "blocks": [
    {
      "block_id": "santiago_aaba0a6b0a2d_block_0",
      "stored_name": "santiago_aaba0a6b0a2d_block_0",
      "size": 1048576,
      "checksum": "ff6b855f63853f50598c6c30e3befec82bc7c40bf4a87a366815c5bdf02f8f1"
    },
    {
      "block_id": "santiago_aaba0a6b0a2d_block_1",
      "stored_name": "santiago_aaba0a6b0a2d_block_1",
      "size": 1048576,
      "checksum": "2c713116fe65f78bf374019067af63f94d8636d59ac10901a96426ede149f94f"
    }
  ]
}
```

```
(.venv) PS C:\Users\santy\Desktop\SISTEMAS\SEXTO SEMESTRE\Nube y sistemas distribuidos\PROYECTO 1\dfs-bloques\dfs-bloques> Invoke-RestMethod http://localhost:8003/blocks | ConvertTo-Json -Depth 10
{
  "node_id": "dn3",
  "count": 2,
  "blocks": [
    {
      "block_id": "santiago_aaba0a6b0a2d_block_1",
      "stored_name": "santiago_aaba0a6b0a2d_block_1",
      "size": 1048576,
      "checksum": "2c713116fe65f78bf374019067af63f94d8636d59ac10901a96426ede149f94f"
    },
    {
      "block_id": "santiago_aaba0a6b0a2d_block_2",
      "stored_name": "santiago_aaba0a6b0a2d_block_2",
      "size": 1048576,
      "checksum": "2c90e7cb6c38ec89e0a7b8d22ebb1a320964ecc046e3c848b1bfc52178382575"
    }
  ]
}
```

Análisis esperado: la prueba debe demostrar que el archivo no se almacena como una sola unidad, sino que se divide en bloques y cada bloque queda replicado en dos DataNodes distintos.

16.6 Prueba ls

```
(.venv) PS C:\Users\santy\Desktop\SISTEMAS\SEXTO SEMESTRE\Nube y sistemas distribuidos\PROYECTO 1\dfs-bloques\dfs-bloques> python client/cli.py ls
{
  "owner": "santiago",
  "directories": [
    "/"
  ],
  "files": [
    {
      "filename": "demo.bin",
      "owner": "santiago",
      "size": 3145728,
      "block_size": 1048576,
      "total_blocks": 3,
      "file_checksum": "22bade239b0df05e2683496736237bcdcfceaab43a15e56c8fb9b215d70a03c5e",
      "replication_factor": 2,
      "status": "available",
      "created_at": 1779470863.4190936,
      "blocks": [
        {
          "block_id": "santiago_aaba0a6b0a2d_block_0",
          "order": 0,
          "size": 1048576,
          "checksum": "ff6b855f63853f50598c6c30e3befec82bc7c40bf4a87a366815c5bdf02f8f1",
          "locations": [
            {
              "node_id": "dn1",
              "client_url": "http://localhost:8001",
              "base_url": "http://datanode1:8001"
            },
            {
              "node_id": "dn2",
              "client_url": "http://localhost:8002",
              "base_url": "http://datanode2:8002"
            }
          ]
        },
        {
          "block_id": "santiago_aaba0a6b0a2d_block_1",

```

```

    ],
    {
      "block_id": "santiago_aaba0a6b0a2d_block_1",
      "order": 1,
      "size": 1048576,
      "checksum": "2c713116fe65f78bf374019067af63f94d8636d59ac10901a9642ede149f94f",
      "locations": [
        {
          "node_id": "dn2",
          "client_url": "http://localhost:8002",
          "base_url": "http://datanode2:8002"
        },
        {
          "node_id": "dn3",
          "client_url": "http://localhost:8003",
          "base_url": "http://datanode3:8003"
        }
      ]
    },
    {
      "block_id": "santiago_aaba0a6b0a2d_block_2",
      "order": 2,
      "size": 1048576,
      "checksum": "2c90e7cb6c38ec89e0a7b8d22eb1a320964ecc046e3c848b1bfc52178382575",
      "locations": [
        {
          "node_id": "dn3",
          "client_url": "http://localhost:8003",
          "base_url": "http://datanode3:8003"
        },
        {
          "node_id": "dn1",
          "client_url": "http://localhost:8001",
          "base_url": "http://datanode1:8001"
        }
      ]
    }
  ]
}

```

16.7 Prueba get y reconstrucción

```

(.venv) PS C:\Users\santy\Desktop\SISTEMAS\SEXTO SEMESTRE\Nube y sistemas distribuidos\PROYECTO 1\dfs-bloques\dfs-bloques> python client/cli.py get demo.bin
--output downloads/demo_recuperado.bin
{
  "message": "Archivo descargado y reconstruido correctamente.",
  "filename": "demo.bin",
  "output": "downloads\\demo_recuperado.bin",
  "sha256": "22bade239b0df05e2683496736237bcdcfceaab43a15e56c8fb9b215d70a03c5e",
  "checksum_match": true,
  "attempted": [
    {
      "block_id": "santiago_aaba0a6b0a2d_block_0",
      "node_id": "dn1",
      "status": 200
    },
    {
      "block_id": "santiago_aaba0a6b0a2d_block_1",
      "node_id": "dn2",
      "status": 200
    },
    {
      "block_id": "santiago_aaba0a6b0a2d_block_2",
      "node_id": "dn3",
      "status": 200
    }
  ]
}

```

```

(.venv) PS C:\Users\santy\Desktop\SISTEMAS\SEXTO SEMESTRE\Nube y sistemas distribuidos\PROYECTO 1\dfs-bloques\dfs-bloques> Get-ChildItem downloads

Directorio: C:\Users\santy\Desktop\SISTEMAS\SEXTO SEMESTRE\Nube y sistemas distribuidos\PROYECTO 1\dfs-bloques\dfs-bloques\downloads

Mode                LastWriteTime         Length Name
----                -
-a-----      22/05/2026   12:32 p. m.       3145728 demo_recuperado.bin

```

Análisis esperado: el cliente consulta los metadatos del NameNode, descarga los bloques desde DataNodes disponibles y reconstruye el archivo en el orden correcto.

16.8 Validación por hash SHA-256

```

(.venv) PS C:\Users\santy\Desktop\SISTEMAS\SEXTO SEMESTRE\Nube y sistemas distribuidos\PROYECTO 1\dfs-bloques\dfs-bloques> Get-FileHash tests\sample_files\demo.bin -Algorithm SHA256

Algorithm Hash Path
-----
SHA256 22BADE239B0DF05E2683496736237BCDFCEAAB43A15E56C8FB9B215D70A03C5E C:\Users\santy\Desktop\SISTEMAS\SEXTO SEMESTRE\Nube y sistemas di...

(.venv) PS C:\Users\santy\Desktop\SISTEMAS\SEXTO SEMESTRE\Nube y sistemas distribuidos\PROYECTO 1\dfs-bloques\dfs-bloques> Get-FileHash downloads\demo_recuperado.bin -Algorithm SHA256

Algorithm Hash Path
-----
SHA256 22BADE239B0DF05E2683496736237BCDFCEAAB43A15E56C8FB9B215D70A03C5E C:\Users\santy\Desktop\SISTEMAS\SEXTO SEMESTRE\Nube y sistemas di...

(.venv) PS C:\Users\santy\Desktop\SISTEMAS\SEXTO SEMESTRE\Nube y sistemas distribuidos\PROYECTO 1\dfs-bloques\dfs-bloques> |

```

Análisis esperado: si ambos hashes son iguales, se confirma que la reconstrucción conserva la integridad del archivo original.

16.9 Caída de un DataNode y recuperación por réplica

```
(.venv) PS C:\Users\santy\Desktop\SISTEMAS\SEXTO SEMESTRE\Nube y sistemas distribuidos\PROYECTO 1\dfs-bloques\dfs-bloques> docker stop dfs-datanode1
dfs-datanode1
(.venv) PS C:\Users\santy\Desktop\SISTEMAS\SEXTO SEMESTRE\Nube y sistemas distribuidos\PROYECTO 1\dfs-bloques\dfs-bloques> docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS                               NAMES
5626a5894f93   dfs-bloques-datanode3              "uvicorn datanode.ma-"   17 minutes ago Up 17 minutes 0.0.0.0:8003->8003/tcp, [::]:8003->8003/tcp   dfs-datanode3
2d3c737efaad   dfs-bloques-datanode2              "uvicorn datanode.ma-"   17 minutes ago Up 17 minutes 0.0.0.0:8002->8002/tcp, [::]:8002->8002/tcp   dfs-datanode2
922ade38552e   dfs-bloques-namenode               "uvicorn namenode.ma-"   17 minutes ago Up 17 minutes 0.0.0.0:8000->8000/tcp, [::]:8000->8000/tcp   dfs-namenode
(.venv) PS C:\Users\santy\Desktop\SISTEMAS\SEXTO SEMESTRE\Nube y sistemas distribuidos\PROYECTO 1\dfs-bloques\dfs-bloques> |
```

```
(.venv) PS C:\Users\santy\Desktop\SISTEMAS\SEXTO SEMESTRE\Nube y sistemas distribuidos\PROYECTO 1\dfs-bloques\dfs-bloques> python client/cli.py get demo.bin
--output downloads/demo_recuperado_fallo.bin
{
  "message": "Archivo descargado y reconstruido correctamente.",
  "filename": "demo.bin",
  "output": "downloads\\demo_recuperado_fallo.bin",
  "sha256": "22bade239b0df05e2683496736237bcdcfceaab43a15e56c8fb9b215d70a03c5e",
  "checksum_match": true,
  "attempted": [
    {
      "block_id": "santiago_aaba0a6b0a2d_block_0",
      "node_id": "dn1",
      "error": "HTTPConnectionPool(host='localhost', port=8001): Max retries exceeded with url: /blocks/santiago_aaba0a6b0a2d_block_0 (Caused by NewConnectionError(\\"HTTPConnection(host='localhost', port=8001): Failed to establish a new connection: [WinError 10061] No se puede establecer una conexi3n ya que el equipo de destino deneg3 expresamente dicha conexi3n\\"))"
    },
    {
      "block_id": "santiago_aaba0a6b0a2d_block_0",
      "node_id": "dn2",
      "status": 200
    },
    {
      "block_id": "santiago_aaba0a6b0a2d_block_1",
      "node_id": "dn2",
      "status": 200
    },
    {
      "block_id": "santiago_aaba0a6b0a2d_block_2",
      "node_id": "dn3",
      "status": 200
    }
  ]
}
```

```
(.venv) PS C:\Users\santy\Desktop\SISTEMAS\SEXTO SEMESTRE\Nube y sistemas distribuidos\PROYECTO 1\dfs-bloques\dfs-bloques> Get-FileHash tests\sample_files\demo.bin -Algorithm SHA256

Algorithm      Hash                                                                 Path
-----
SHA256         22BADE239B0DF05E2683496736237BCDFCEAAB43A15E56C8FB9B215D70A03C5E C:\Users\santy\Desktop\SISTEMAS\SEXTO SEMESTRE\Nube y sistemas di...

(.venv) PS C:\Users\santy\Desktop\SISTEMAS\SEXTO SEMESTRE\Nube y sistemas distribuidos\PROYECTO 1\dfs-bloques\dfs-bloques> Get-FileHash downloads\demo_recuperado_fallo.bin -Algorithm SHA256

Algorithm      Hash                                                                 Path
-----
SHA256         22BADE239B0DF05E2683496736237BCDFCEAAB43A15E56C8FB9B215D70A03C5E C:\Users\santy\Desktop\SISTEMAS\SEXTO SEMESTRE\Nube y sistemas di...

(.venv) PS C:\Users\santy\Desktop\SISTEMAS\SEXTO SEMESTRE\Nube y sistemas distribuidos\PROYECTO 1\dfs-bloques\dfs-bloques> |
```

```
(.venv) PS C:\Users\santy\Desktop\SISTEMAS\SEXTO SEMESTRE\Nube y sistemas distribuidos\PROYECTO 1\dfs-bloques\dfs-bloques> docker compose logs --tail=100
dfs-namenode INFO: Started server process [1]
dfs-namenode INFO: Waiting for application startup.
dfs-namenode INFO: Application startup complete.
dfs-namenode INFO: Uvicorn running on http://0.0.0.0:8000 (Press CTRL+C to quit)
dfs-namenode INFO: 172.18.0.3:42636 - "POST /datanodes/register HTTP/1.1" 200 OK
dfs-namenode INFO: 172.18.0.5:38150 - "POST /datanodes/register HTTP/1.1" 200 OK
dfs-namenode INFO: 172.18.0.4:34046 - "POST /datanodes/register HTTP/1.1" 200 OK
dfs-namenode INFO: 172.18.0.1:43974 - "GET /docs HTTP/1.1" 200 OK
dfs-namenode INFO: 172.18.0.1:43974 - "GET /openapi.json HTTP/1.1" 200 OK
dfs-namenode INFO: 172.18.0.1:58950 - "GET /health HTTP/1.1" 200 OK
dfs-namenode INFO: 172.18.0.1:48012 - "GET /datanodes HTTP/1.1" 200 OK
dfs-datanode2 INFO: Started server process [1]
dfs-datanode2 INFO: Waiting for application startup.
dfs-datanode2 INFO: Application startup complete.
dfs-datanode2 INFO: Uvicorn running on http://0.0.0.0:8002 (Press CTRL+C to quit)
dfs-datanode2 [dn2] Registrado en NameNode: http://namenode:8000
dfs-datanode2 INFO: 172.18.0.1:41970 - "GET /docs HTTP/1.1" 200 OK
dfs-datanode2 INFO: 172.18.0.1:41970 - "GET /openapi.json HTTP/1.1" 200 OK
dfs-datanode2 INFO: 172.18.0.2:59648 - "GET /health HTTP/1.1" 200 OK
dfs-datanode2 INFO: 172.18.0.2:54840 - "GET /health HTTP/1.1" 200 OK
dfs-datanode2 INFO: 172.18.0.1:33330 - "POST /blocks/santiago_aaba0a6b0a2d_block_0 HTTP/1.1" 200 OK
dfs-datanode2 INFO: 172.18.0.1:33336 - "POST /blocks/santiago_aaba0a6b0a2d_block_1 HTTP/1.1" 200 OK
dfs-datanode2 INFO: 172.18.0.1:53154 - "GET /blocks HTTP/1.1" 200 OK
dfs-datanode2 INFO: 172.18.0.1:44208 - "GET /blocks/santiago_aaba0a6b0a2d_block_1 HTTP/1.1" 200 OK
dfs-datanode2 INFO: 172.18.0.1:47800 - "GET /blocks/santiago_aaba0a6b0a2d_block_0 HTTP/1.1" 200 OK
dfs-datanode2 INFO: 172.18.0.1:47806 - "GET /blocks/santiago_aaba0a6b0a2d_block_1 HTTP/1.1" 200 OK
dfs-namenode INFO: 172.18.0.1:48426 - "POST /auth/login HTTP/1.1" 200 OK
dfs-namenode INFO: 172.18.0.1:37000 - "POST /auth/login HTTP/1.1" 401 Unauthorized
dfs-namenode INFO: 172.18.0.1:37338 - "POST /auth/login HTTP/1.1" 200 OK
dfs-namenode INFO: 172.18.0.1:39686 - "POST /files/put-request HTTP/1.1" 200 OK
dfs-namenode INFO: 172.18.0.1:39688 - "POST /files/commit HTTP/1.1" 200 OK
dfs-namenode INFO: 172.18.0.1:55520 - "GET /files HTTP/1.1" 200 OK
dfs-namenode INFO: 172.18.0.1:34234 - "GET /files/demo.bin/blocks HTTP/1.1" 200 OK
dfs-namenode INFO: 172.18.0.1:43580 - "GET /files/demo.bin/blocks HTTP/1.1" 200 OK
dfs-datanode3 INFO: Started server process [1]
dfs-datanode3 INFO: Waiting for application startup.
dfs-datanode3 INFO: Application startup complete.
dfs-datanode3 INFO: Uvicorn running on http://0.0.0.0:8003 (Press CTRL+C to quit)
```

```
dfs-namenode INFO: 172.18.0.1:43580 - "GET /files/demo.bin/blocks HTTP/1.1" 200 OK
dfs-datanode3 INFO: Started server process [1]
dfs-datanode3 INFO: Waiting for application startup.
dfs-datanode3 INFO: Application startup complete.
dfs-datanode3 INFO: Uvicorn running on http://0.0.0.0:8003 (Press CTRL+C to quit)
dfs-datanode3 [dn3] Registrado en NameNode: http://namenode:8000
dfs-datanode3 INFO: 172.18.0.1:57966 - "GET /docs HTTP/1.1" 200 OK
dfs-datanode3 INFO: 172.18.0.1:57966 - "GET /openapi.json HTTP/1.1" 200 OK
dfs-datanode3 INFO: 172.18.0.2:39788 - "GET /health HTTP/1.1" 200 OK
dfs-datanode3 INFO: 172.18.0.2:52758 - "GET /health HTTP/1.1" 200 OK
dfs-datanode3 INFO: 172.18.0.1:38942 - "POST /blocks/santiago_aaba0a6b0a2d_block_1 HTTP/1.1" 200 OK
dfs-datanode3 INFO: 172.18.0.1:38946 - "POST /blocks/santiago_aaba0a6b0a2d_block_2 HTTP/1.1" 200 OK
dfs-datanode3 INFO: 172.18.0.1:49270 - "GET /blocks HTTP/1.1" 200 OK
dfs-datanode3 INFO: 172.18.0.1:39152 - "GET /blocks/santiago_aaba0a6b0a2d_block_2 HTTP/1.1" 200 OK
dfs-datanode3 INFO: 172.18.0.1:35188 - "GET /blocks/santiago_aaba0a6b0a2d_block_2 HTTP/1.1" 200 OK
dfs-datanode1 INFO: Started server process [1]
dfs-datanode1 INFO: Waiting for application startup.
dfs-datanode1 INFO: Application startup complete.
dfs-datanode1 INFO: Uvicorn running on http://0.0.0.0:8001 (Press CTRL+C to quit)
dfs-datanode1 [dn1] Registrado en NameNode: http://namenode:8000
dfs-datanode1 INFO: 172.18.0.1:37674 - "GET /docs HTTP/1.1" 200 OK
dfs-datanode1 INFO: 172.18.0.1:37674 - "GET /openapi.json HTTP/1.1" 200 OK
dfs-datanode1 INFO: 172.18.0.1:37678 - "GET /docs HTTP/1.1" 200 OK
dfs-datanode1 INFO: 172.18.0.1:37678 - "GET /openapi.json HTTP/1.1" 200 OK
dfs-datanode1 INFO: 172.18.0.2:44740 - "GET /health HTTP/1.1" 200 OK
dfs-datanode1 INFO: 172.18.0.2:54896 - "GET /health HTTP/1.1" 200 OK
dfs-datanode1 INFO: 172.18.0.1:44750 - "POST /blocks/santiago_aaba0a6b0a2d_block_0 HTTP/1.1" 200 OK
dfs-datanode1 INFO: 172.18.0.1:44758 - "POST /blocks/santiago_aaba0a6b0a2d_block_2 HTTP/1.1" 200 OK
dfs-datanode1 INFO: 172.18.0.1:57298 - "GET /blocks HTTP/1.1" 200 OK
dfs-datanode1 INFO: 172.18.0.1:43750 - "GET /blocks/santiago_aaba0a6b0a2d_block_0 HTTP/1.1" 200 OK
dfs-datanode1 INFO: Shutting down
dfs-datanode1 INFO: Waiting for application shutdown.
dfs-datanode1 INFO: Application shutdown complete.
dfs-datanode1 INFO: Finished server process [1]
(.venv) PS C:\Users\santy\Desktop\SISTEMAS\SEXTO SEMESTRE\Nube y sistemas distribuidos\PROYECTO 1\dfs-bloques\dfs-bloques>
```

Análisis esperado: el cliente debe intentar recuperar el bloque desde el DataNode caído, registrar el error de conexión y luego descargarlo desde otra réplica disponible. Esto evidencia tolerancia básica a fallos.

16.10 Prueba rm

```
(.venv) PS C:\Users\santy\Desktop\SISTEMAS\SEXTO SEMESTRE\Nube y sistemas distribuidos\PROYECTO 1\dfs-bloques\dfs-bloques> python client/cli.py rm demo.bin
{
  "message": "Archivo eliminado de metadatos. Se intentó eliminar cada réplica física.",
  "file_removed": true,
  "deleted_blocks": [
    {
      "block_id": "santiago_aaba0a6b0a2d_block_0",
      "node_id": "dn1"
    },
    {
      "block_id": "santiago_aaba0a6b0a2d_block_0",
      "node_id": "dn2"
    },
    {
      "block_id": "santiago_aaba0a6b0a2d_block_1",
      "node_id": "dn2"
    },
    {
      "block_id": "santiago_aaba0a6b0a2d_block_1",
      "node_id": "dn3"
    },
    {
      "block_id": "santiago_aaba0a6b0a2d_block_2",
      "node_id": "dn3"
    },
    {
      "block_id": "santiago_aaba0a6b0a2d_block_2",
      "node_id": "dn1"
    }
  ],
  "failed_deletions": []
}
```

```
(.venv) PS C:\Users\santy\Desktop\SISTEMAS\SEXTO SEMESTRE\Nube y sistemas distribuidos\PROYECTO 1\dfs-bloques\dfs-bloques> python client/cli.py ls
{
  "owner": "santiago",
  "directories": [
    "/"
  ],
  "files": []
}
```

```
(.venv) PS C:\Users\santy\Desktop\SISTEMAS\SEXTO SEMESTRE\Nube y sistemas distribuidos\PROYECTO 1\dfs-bloques\dfs-bloques> python client/cli.py get demo.bin
--output downloads/demo_no_deberia_descargar.bin
ERROR: HTTP 404: Archivo no encontrado.
```

16.11 Prueba mkdir y rmdir

```
(.venv) PS C:\Users\santy\Desktop\SISTEMAS\SEXTO SEMESTRE\Nube y sistemas distribuidos\PROYECTO 1\dfs-bloques\dfs-bloques> python client/cli.py mkdir documentos
{
  "message": "Directorio creado correctamente.",
  "path": "documentos"
}
(.venv) PS C:\Users\santy\Desktop\SISTEMAS\SEXTO SEMESTRE\Nube y sistemas distribuidos\PROYECTO 1\dfs-bloques\dfs-bloques> python client/cli.py ls
{
  "owner": "santiago",
  "directories": [
    "/",
    "/documentos"
  ],
  "files": []
}
(.venv) PS C:\Users\santy\Desktop\SISTEMAS\SEXTO SEMESTRE\Nube y sistemas distribuidos\PROYECTO 1\dfs-bloques\dfs-bloques> python client/cli.py rmdir documentos
{
  "message": "Directorio eliminado correctamente.",
  "path": "documentos"
}
(.venv) PS C:\Users\santy\Desktop\SISTEMAS\SEXTO SEMESTRE\Nube y sistemas distribuidos\PROYECTO 1\dfs-bloques\dfs-bloques> python client/cli.py ls
{
  "owner": "santiago",
  "directories": [
    "/"
  ],
  "files": []
}
```

16.12 Evidencias de repositorio y gestión

Proyecto-Sistemas-Distribuidos- Public

Pin Watch 0 Fork 0 Star 0

main 1 Branch 0 Tags

Go to file

Add file

Code

josuepinop Update README.md ae8af44 · 8 minutes ago 15 Commits

client	Add files via upload	23 minutes ago
common	Add files via upload	17 minutes ago
datanode	Add files via upload	17 minutes ago
namenode	Add files via upload	17 minutes ago
scripts	Add files via upload	23 minutes ago
tests	Add files via upload	23 minutes ago
Dockerfile	Add files via upload	16 minutes ago
README.md	Update README.md	8 minutes ago
docker-compose.yml	Add files via upload	16 minutes ago
requirements.txt	Add files via upload	16 minutes ago

README

Proyecto Sistemas Distribuidos - DFS-Bloques

Sistema de archivos distribuido minimalista basado en bloques, desarrollado para el curso **Arquitecturas de Nube y Sistemas Distribuidos**.

El proyecto implementa un DFS académico compuesto por un **NameNode**, tres **DataNodes** y un **cliente CLI**. Permite subir, dividir, distribuir, replicar, listar, recuperar y eliminar archivos usando comunicación REST sobre HTTP.

About

No description, website, or topics provided.

Readme

Activity

0 stars

0 watching

0 forks

Releases

No releases published

Create a new release

Packages

No packages published

Publish your first package

Contributors 2

josuepinop

santiagoretrepo14 Santiago Restre...

Languages

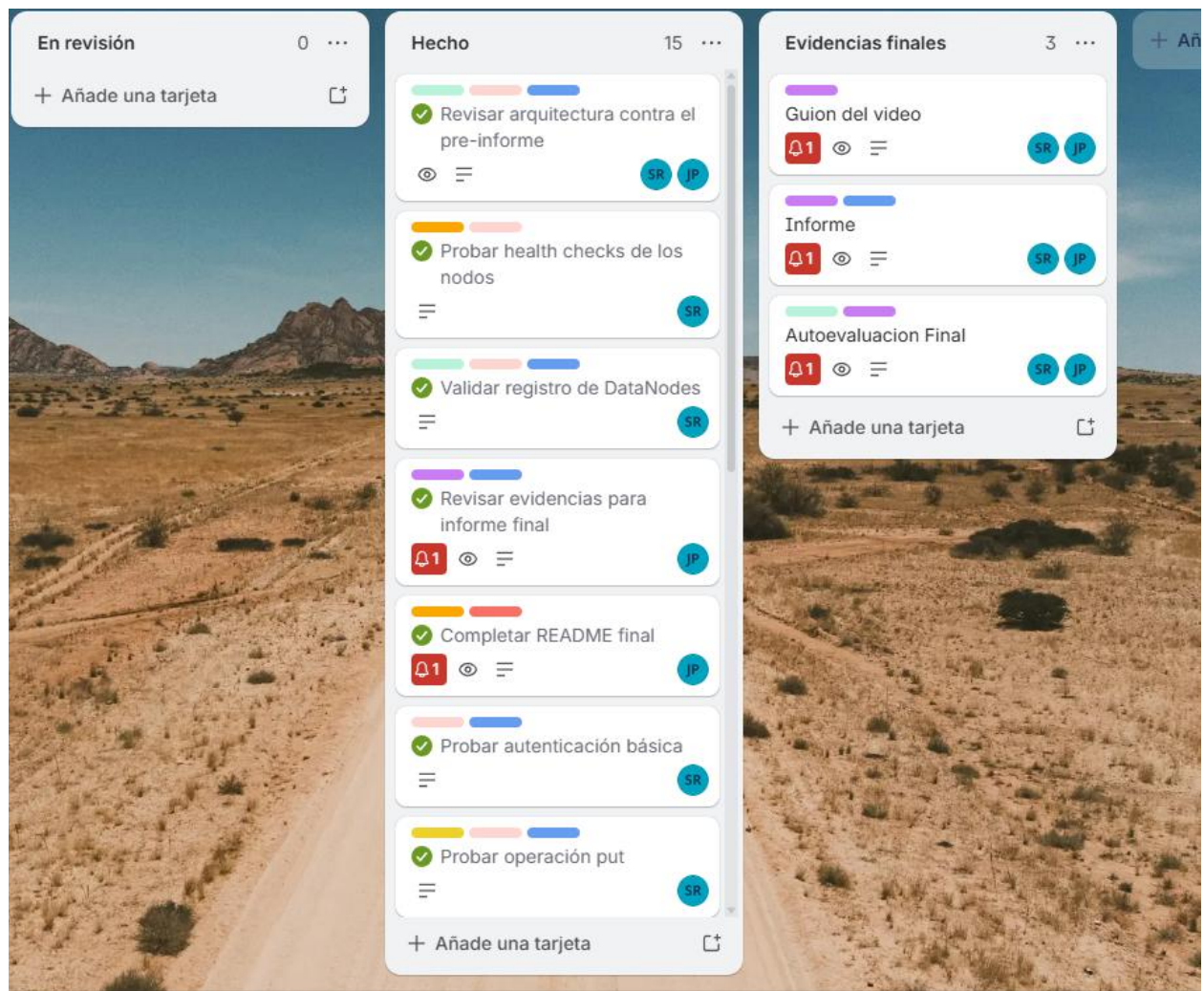
Python 97.1%

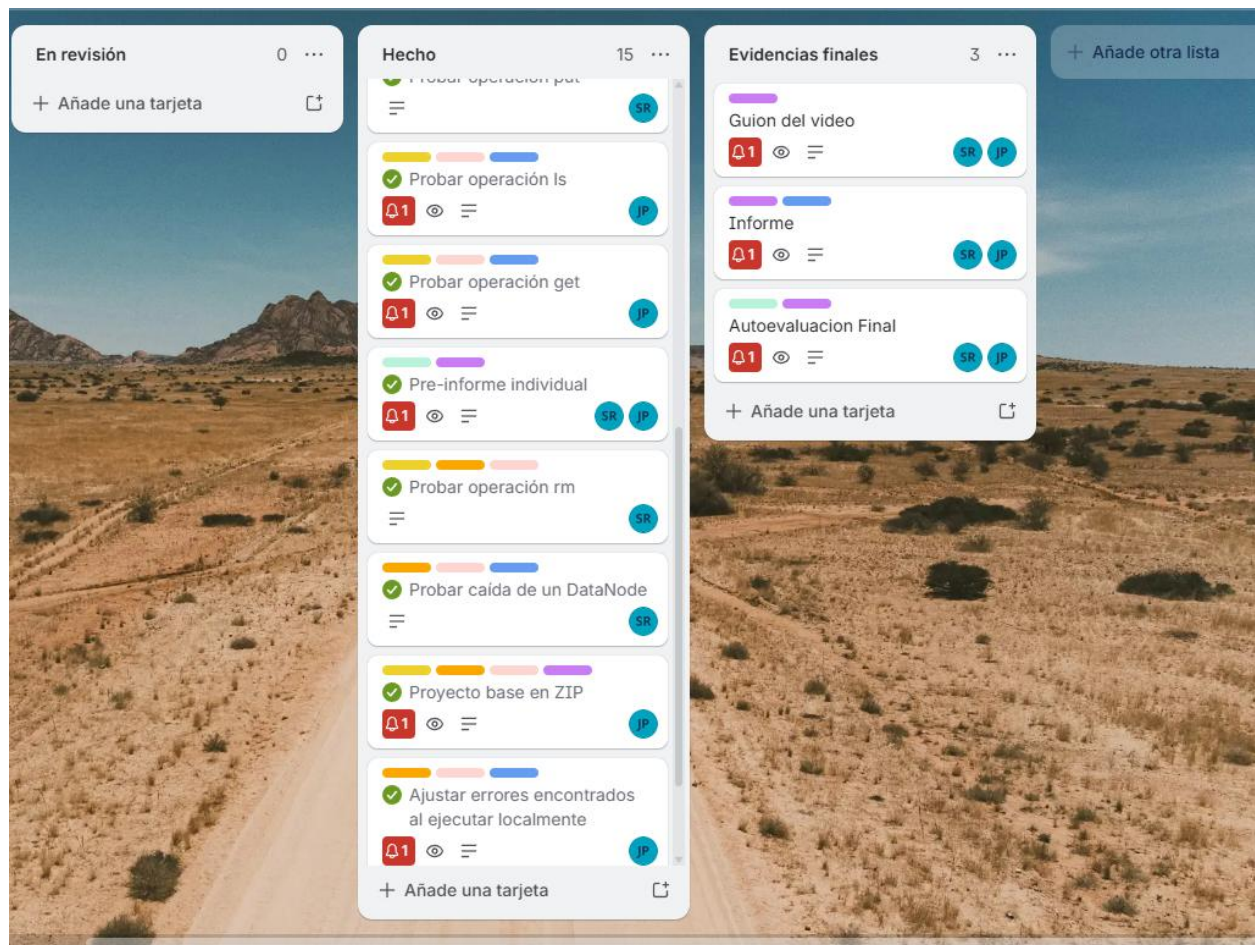
Shell 1.1%

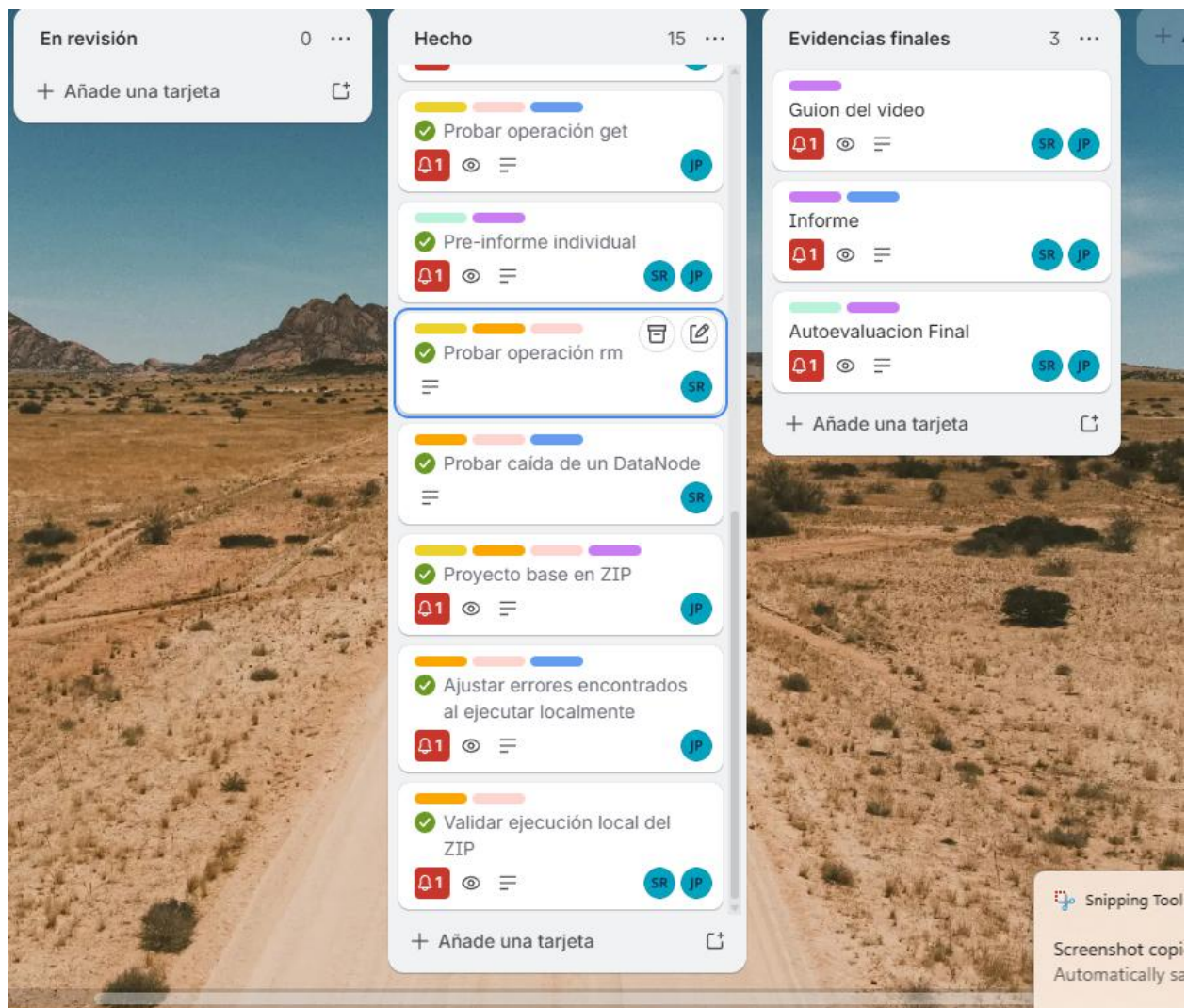
Other 1.8%

Suggested workflows

Based on your tech stack







17. Problemas encontrados y ajustes realizados

Durante la validación final del sistema DFS-Bloques no se presentaron errores críticos que impidieran ejecutar el flujo principal del proyecto. El sistema logró levantar correctamente el NameNode y los tres DataNodes mediante Docker Compose, registrar los nodos de almacenamiento, autenticar usuarios, subir archivos, dividirlos en bloques, distribuirlos, replicarlos, descargarlos, validar su integridad mediante SHA-256 y recuperar archivos aun cuando uno de los DataNodes fue detenido de forma intencional.

Los inconvenientes encontrados estuvieron relacionados principalmente con el entorno de ejecución local, la preparación de evidencias y las pruebas controladas de tolerancia a fallos. No fue necesario modificar la arquitectura general del sistema, ya que los ajustes realizados correspondieron a configuración, ejecución de comandos y validación del comportamiento esperado.

Problema encontrado	Causa probable	Ajuste realizado	Estado

Docker no respondía inicialmente al ejecutar comandos como docker ps.	Docker Desktop no estaba iniciado o el motor de Docker no se encontraba activo.	Se abrió Docker Desktop, se esperó el inicio del motor de ejecución y se volvió a validar el comando docker ps.	Solucionado
Se requería evidenciar claramente el particionamiento del archivo en varios bloques.	Con el tamaño de bloque por defecto de 64 MB, un archivo pequeño podía almacenarse como un único bloque, dificultando la evidencia visual.	Se generó un archivo de prueba demo.bin de 3 MB y se ejecutó la operación put usando un tamaño de bloque de 1 MB. Esto permitió obtener tres bloques visibles.	Solucionado
Era necesario demostrar la distribución física de bloques entre DataNodes.	La distribución no se evidencia únicamente con la operación put; también era necesario consultar los nodos de almacenamiento.	Se consultaron los endpoints /blocks de los DataNodes en los puertos 8001, 8002 y 8003, verificando que los bloques estaban distribuidos y replicados.	Validado
Se simuló la caída de un DataNode durante la prueba de tolerancia a fallos.	Se detuvo intencionalmente el contenedor dfs-datanode1 para comprobar si el sistema podía recuperar el archivo desde otra réplica.	Se ejecutó docker stop dfs-datanode1 y posteriormente se realizó una operación get. El cliente detectó el fallo de conexión y continuó con otra réplica disponible.	Validado
El cliente intentó leer inicialmente desde un DataNode detenido.	El NameNode conservaba la ubicación de la réplica en el DataNode detenido, pero el cliente debía probar las demás ubicaciones disponibles.	El cliente registró el error de conexión y luego descargó el bloque desde otro DataNode disponible. La reconstrucción final fue correcta.	Validado
Después de eliminar un archivo, el sistema no permitía descargarlo nuevamente.	Este comportamiento era esperado, ya que la operación rm elimina los metadatos del archivo y las réplicas físicas asociadas.	Se verificó que el sistema respondiera con un error controlado HTTP 404: Archivo no encontrado, confirmando que la eliminación fue efectiva.	Validado
Ejecución de comandos desde	El proyecto fue ejecutado desde un entorno Windows con Docker Desktop, por lo que era necesario cuidar rutas,	Se trabajó desde la carpeta raíz del proyecto, se activó el entorno virtual .venv y se	Solucionado

PowerShell en Windows.	activación del entorno virtual y ubicación correcta en la carpeta raíz.	ejecutaron los comandos desde PowerShell.	
------------------------	---	---	--

En general, las pruebas confirmaron que la implementación funciona correctamente en el entorno Docker local definido para la demostración. Los errores observados fueron controlados y esperados dentro del proceso de validación, especialmente en la prueba de caída de un DataNode y en el intento de descargar un archivo previamente eliminado.

18. Conclusiones

El proyecto DFS-Bloques permitió diseñar e implementar un sistema de archivos distribuido minimalista basado en bloques. La solución desarrollada separa claramente el plano de control y el plano de datos: el NameNode administra metadatos y asignaciones, mientras que los DataNodes almacenan físicamente los bloques y el cliente ejecuta las operaciones de carga, descarga y gestión.

La arquitectura Cliente/Servidor con enfoque Maestro-Trabajador resultó adecuada para el alcance académico del proyecto. El NameNode centraliza la información de usuarios, directorios, archivos, bloques y ubicaciones, mientras que los DataNodes funcionan como nodos independientes de almacenamiento. Esta separación permitió demostrar cómo un archivo lógico puede estar distribuido físicamente en varios nodos.

La operación put validó el particionamiento de archivos en bloques configurables. Para las pruebas se utilizó un archivo demo.bin de 3 MB con tamaño de bloque de 1 MB, lo que permitió observar tres bloques independientes. Cada bloque fue replicado en dos DataNodes distintos, cumpliendo el factor de replicación definido y evidenciando la distribución física de los datos.

La operación get permitió reconstruir correctamente el archivo a partir de los bloques almacenados en los DataNodes. La comparación SHA-256 entre el archivo original y el archivo recuperado confirmó que la reconstrucción mantuvo la integridad de los datos. Esto demuestra que el sistema conserva el orden correcto de los bloques y que los metadatos del NameNode son suficientes para recuperar el archivo.

La prueba de caída de DataNode permitió validar una tolerancia básica a fallos. Al detener el contenedor dfs-datanode1, el cliente intentó acceder a una réplica no disponible, registró el error de conexión y posteriormente recuperó el bloque desde otro DataNode disponible. El hash del archivo reconstruido durante la falla coincidió con el original, confirmando que la replicación permite mantener disponibilidad mientras exista al menos una copia sana de cada bloque.

Las operaciones `rm`, `mkdir` y `rmdir` permitieron validar la gestión básica de archivos y directorios solicitada en el enunciado. La prueba `rm` eliminó los metadatos del archivo y sus réplicas físicas, y el intento posterior de descarga respondió con un error controlado de archivo no encontrado. Las pruebas `mkdir` y `rmdir` confirmaron la creación y eliminación de directorios lógicos asociados al usuario.

El uso de Docker Compose facilitó la simulación de un entorno distribuido local con un NameNode y tres DataNodes como contenedores independientes. Este entorno permitió cumplir el criterio de ejecución en nodos o contenedores, exponer APIs independientes por puerto, consultar Swagger para cada servicio y verificar la comunicación entre componentes mediante REST sobre HTTP.

Como limitación principal, DFS-Bloques no implementa alta disponibilidad del NameNode, consenso distribuido, re-replicación automática completa en segundo plano, integración con el sistema de archivos del sistema operativo ni compatibilidad POSIX. Sin embargo, estas características están fuera del alcance de la versión minimalista y pueden considerarse mejoras futuras.

En conclusión, el sistema cumple el objetivo central del proyecto: permite almacenar y recuperar archivos mediante particionamiento en bloques, distribución entre DataNodes, replicación, metadatos centralizados, CLI funcional, APIs documentadas y tolerancia básica ante fallos. Las evidencias obtenidas en Docker local respaldan el funcionamiento de la solución y permiten demostrar los conceptos fundamentales de un DFS por bloques.

19. Referencias

Ospina Sanjuan, A. (2026). **Proyecto: Sistema de archivos distribuidos por bloques**. Universidad Pontificia Bolivariana, curso Arquitecturas de Nube y Sistemas Distribuidos.

Ospina Sanjuan, A. (2026). **Comunicaciones y Middleware**. Material de clase, Universidad Pontificia Bolivariana.

Ghemawat, S., Gobioff, H., & Leung, S.-T. (2003). **The Google File System**. Proceedings of the 19th ACM Symposium on Operating Systems Principles.

Shvachko, K., Kuang, H., Radia, S., & Chansler, R. (2010). **The Hadoop Distributed File System**. IEEE Symposium on Mass Storage Systems and Technologies.

Fielding, R. T. (2000). **Architectural Styles and the Design of Network-based Software Architectures**. Doctoral dissertation, University of California, Irvine.

FastAPI. (s. f.). **FastAPI Documentation**. Referencia técnica para APIs REST en Python.

Docker. (s. f.). **Docker Compose Documentation**. Referencia técnica para ejecución local de contenedores.

20. Autoevaluación del proyecto

Cumplimiento general del proyecto

El proyecto cumplió con el objetivo principal de diseñar e implementar un sistema de archivos distribuido minimalista por bloques. Se desarrolló una arquitectura compuesta por un cliente CLI, un NameNode central para la gestión de metadatos y tres DataNodes para el almacenamiento físico de bloques. El sistema permite cargar, listar, descargar y eliminar archivos, además de crear y eliminar directorios lógicos.

La validación se realizó en Docker local mediante Docker Compose, ejecutando un NameNode y tres DataNodes como contenedores independientes. Este entorno permitió evidenciar la comunicación entre componentes, la distribución de bloques, la replicación con factor dos y la recuperación de archivos ante la caída de un DataNode.

Cumplimiento de requerimientos funcionales

Se implementaron las operaciones principales solicitadas en el enunciado:

- put: permite subir un archivo, dividirlo en bloques y distribuirlo entre DataNodes.
- get: permite reconstruir un archivo a partir de sus bloques.
- ls: permite listar archivos y directorios del usuario.
- rm: permite eliminar un archivo, sus metadatos y sus réplicas físicas.
- mkdir: permite crear directorios lógicos.
- rmdir: permite eliminar directorios lógicos.
- login: permite validar autenticación básica de usuario y contraseña.

También se verificó el manejo de errores, incluyendo login fallido, intento de descarga de archivo inexistente y recuperación ante DataNode no disponible.

Cumplimiento de arquitectura

La arquitectura implementada corresponde a un modelo Cliente/Servidor combinado con Maestro-Trabajador. El NameNode actúa como componente maestro y administra metadatos, usuarios, directorios, archivos, bloques, ubicaciones de réplicas y estado de DataNodes. Los DataNodes actúan como trabajadores y almacenan bloques físicos. El cliente se comunica con el NameNode para operaciones de control y con los DataNodes para transferencia de datos.

La comunicación se implementó mediante REST sobre HTTP. Los metadatos se transmiten en JSON y los bloques se transfieren como contenido binario. Esta decisión facilitó la documentación automática de APIs mediante Swagger y la validación de los endpoints.

Cumplimiento de particionamiento y replicación

El archivo de prueba demo.bin fue generado con tamaño aproximado de 3 MB y se subió al sistema usando un tamaño de bloque de 1 MB para evidenciar la partición. El sistema dividió el archivo en tres bloques y cada bloque fue replicado en dos DataNodes distintos, cumpliendo el factor de replicación configurado.

La distribución de bloques fue validada mediante los endpoints /blocks de cada DataNode y mediante la salida del comando ls, donde se observaron los metadatos del archivo, los identificadores de bloque, el orden, los checksums y las ubicaciones de réplica.

Cumplimiento de reconstrucción e integridad

La operación get permitió descargar y reconstruir correctamente el archivo demo.bin desde los bloques almacenados en los DataNodes. La salida del cliente indicó checksum_match: true. Además, se comparó el hash SHA-256 del archivo original con el hash SHA-256 del archivo recuperado, obteniendo coincidencia exacta. Esto confirma que la reconstrucción conservó la integridad del archivo.

Cumplimiento de tolerancia básica a fallos

Se detuvo intencionalmente el contenedor dfs-datanode1 para simular la caída de un DataNode. Posteriormente, se ejecutó nuevamente la operación get. El cliente intentó recuperar un bloque desde el DataNode detenido, registró un error de conexión y luego obtuvo el bloque desde otra réplica disponible. El archivo fue reconstruido correctamente y la comparación SHA-256 confirmó su integridad.

Esta prueba evidencia una tolerancia básica a fallos, siempre que exista al menos una réplica disponible por cada bloque.

Gestión del proyecto

El grupo utilizó Trello como herramienta de gestión de tareas. El tablero permitió organizar actividades en listas de avance, asignar responsables, usar etiquetas y hacer seguimiento de tareas como arquitectura, backend, cliente CLI, Docker, pruebas, documentación, evidencias, informe final y video. La captura final del tablero se adjunta como evidencia de gestión.

Entregables

Se prepararon los siguientes entregables:

- Código fuente en repositorio GitHub.

- README con instrucciones de instalación, ejecución y pruebas.
- Informe técnico final.
- Evidencias de ejecución en Docker local.
- Capturas de APIs, CLI, DataNodes, replicación, recuperación y hash.
- Tablero Trello.
- Autoevaluación.
- Video demostrativo.

Limitaciones

La versión implementada es académica y minimalista. No incluye integración POSIX, montaje como sistema de archivos del sistema operativo, alta disponibilidad del NameNode, consenso distribuido, balanceo avanzado, cifrado extremo a extremo ni re-replicación automática completa en segundo plano. Estas características quedan planteadas como mejoras futuras.

Porcentaje de cumplimiento estimado

El proyecto cumple en alto grado los requisitos definidos: arquitectura, particionamiento, replicación, CLI/API, comunicación entre nodos, documentación, pruebas y evidencias. El cumplimiento estimado es del 90% al 95%, dependiendo de la validación final del video, la autoevaluación y la revisión del repositorio por parte del docente.

Reflexión final

El proyecto permitió aplicar conceptos fundamentales de sistemas distribuidos, especialmente separación entre plano de control y plano de datos, comunicación entre procesos, metadatos centralizados, almacenamiento distribuido, replicación y tolerancia básica a fallos. La ejecución con Docker Compose facilitó la simulación de un clúster distribuido en un entorno controlado, permitiendo evidenciar de manera clara el funcionamiento de un DFS por bloques.